

Final Review

Zijun Yao

Assistant Professor, EECS Department
The University of Kansas

Final exam

- Time: May 15 Thursday 4:30pm – 6:30pm (120 minutes)
- Location: LEEP2 2300 (Classroom for lectures)
- Important Instruction:
 - 6 questions in total, only cover topics not included in midterm
 - Open-book - lecture slides, notes, books are allowed. Calculator is allowed. No any other electronic devices (Notes have to be printed)
 - **Write clearly.**
 - **Don't get stuck at particular problems. Watch the time.**
- Good luck!

Agenda

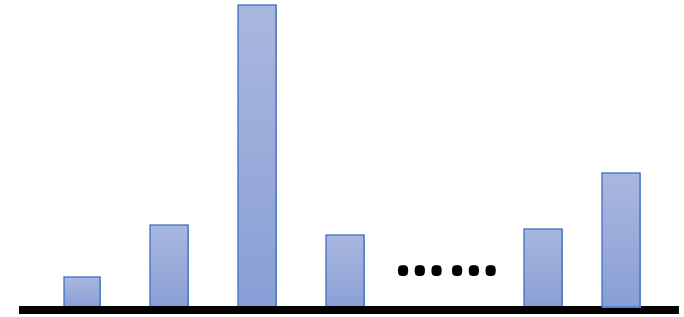
- Transformer Network
- Naïve Bayes Classifier
- K-means Clustering
- Boosting
- Model Generalization
- Evaluation Metrics

Transformer overview

This is a generative AI

Language
Model

Introduction



1.Tokenization

2. Input
Layer

3. Attention

4. Feed
Forward

5. Output
Layer

Word to Token

Understand Token

Understand Context

Aggregate

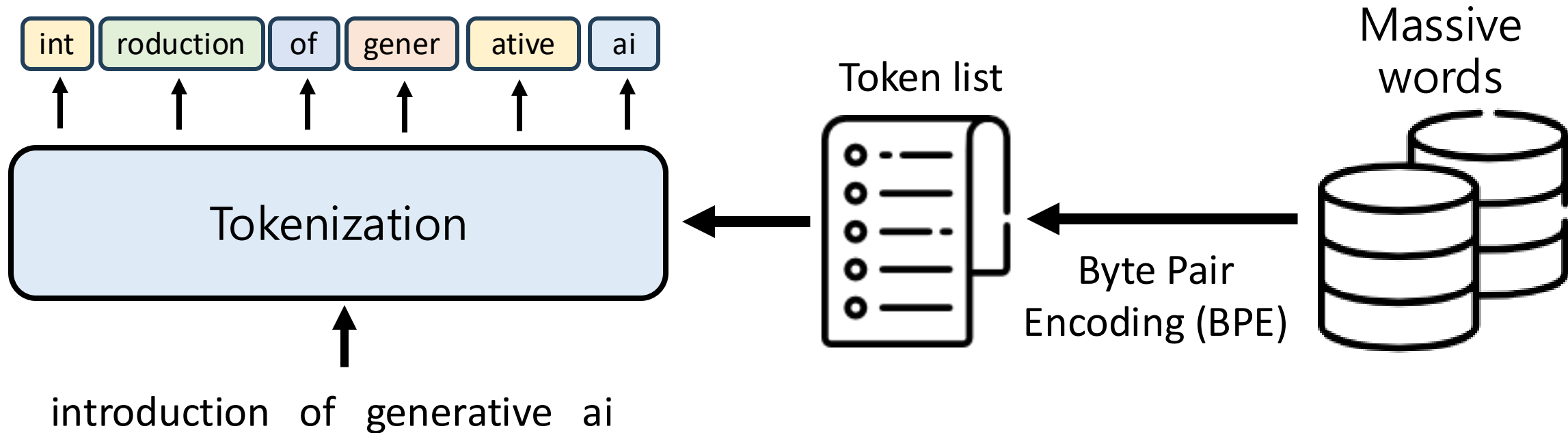
Get
Output

Transformer Block

Step 1: Convert text to tokens

- Language models process text in units called “tokens”

Tokenization has no trainable parameters



GPT-3.5 & GPT-4 GPT-3 (Legacy)

A language model is a probabilistic model of a natural language. In 1980, the first significant statistical language model was proposed, and during the decade IBM performed 'Shannon-style' experiments, in which potential sources for language modeling improvement were identified by observing and analyzing the performance of human subjects in predicting or correcting text.



Tokens	Characters
65	373

A language model is a probabilistic model of a natural language. In 1980, the first significant statistical language model was proposed, and during the decade IBM performed 'Shannon-style' experiments, in which potential sources for language modeling improvement were identified by observing and analyzing the performance of human subjects in predicting or correcting text.

Text Token IDs

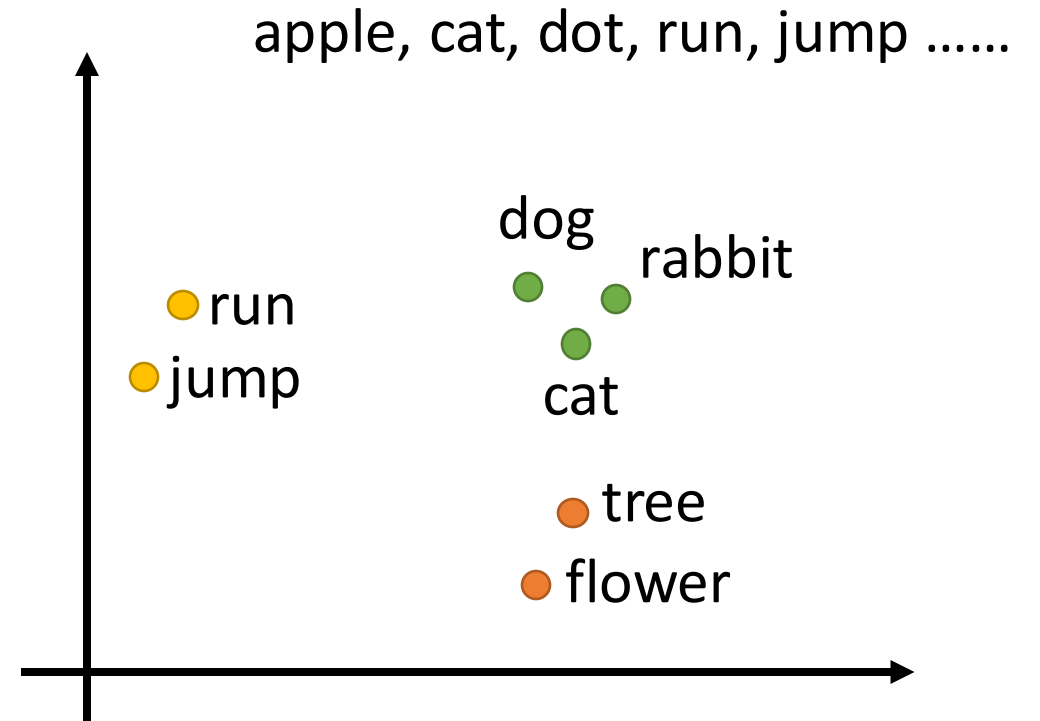
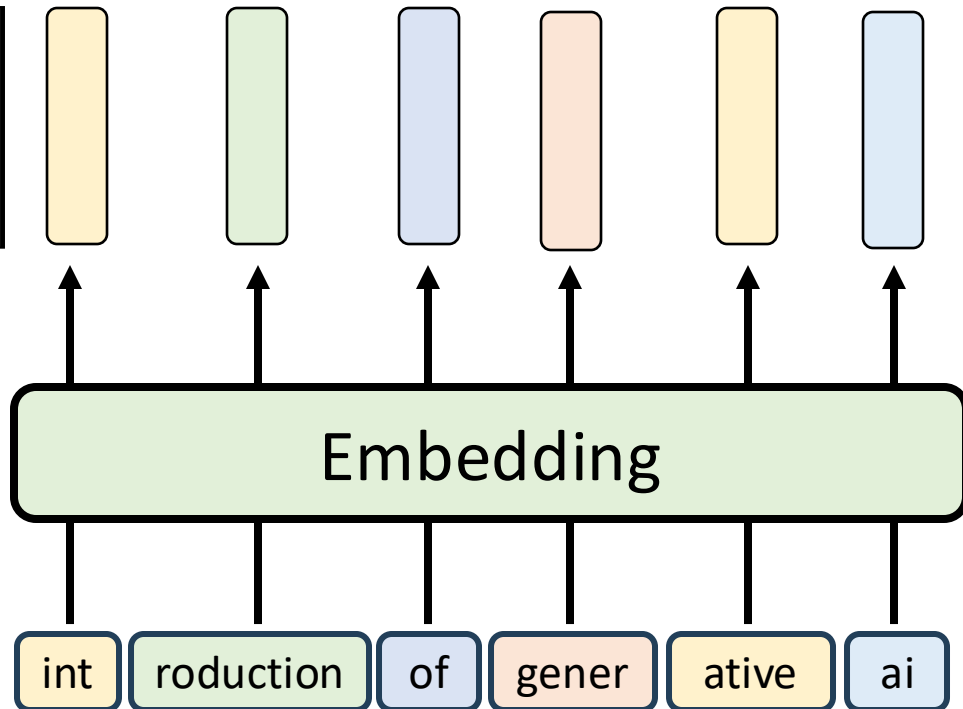
<https://platform.openai.com/tokenizer>

Step 2: Understand each token

Tokens with similar meanings have similar embeddings.

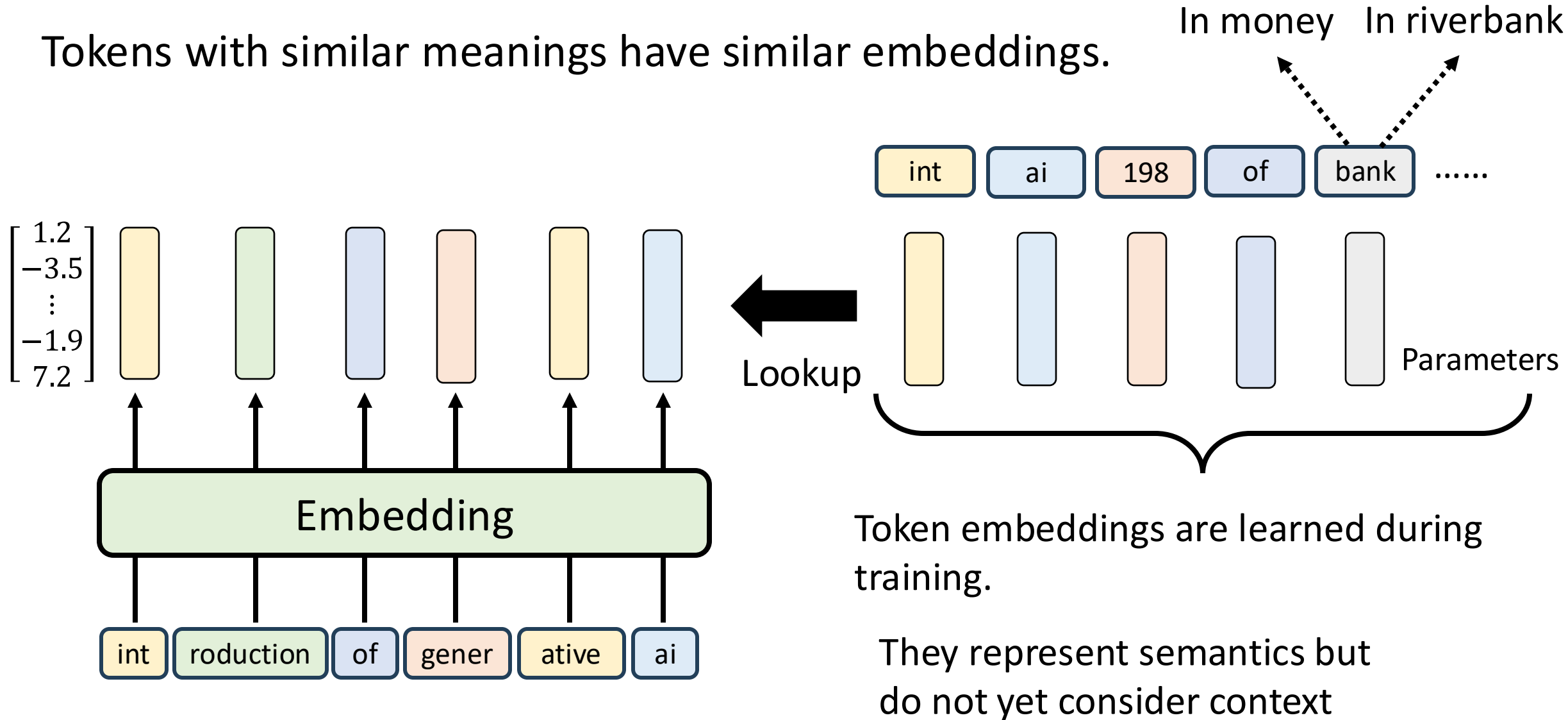
- dog, cat, rabbit → similar vectors
- jump, run → similar vectors
- flower, tree → similar vectors

Vector

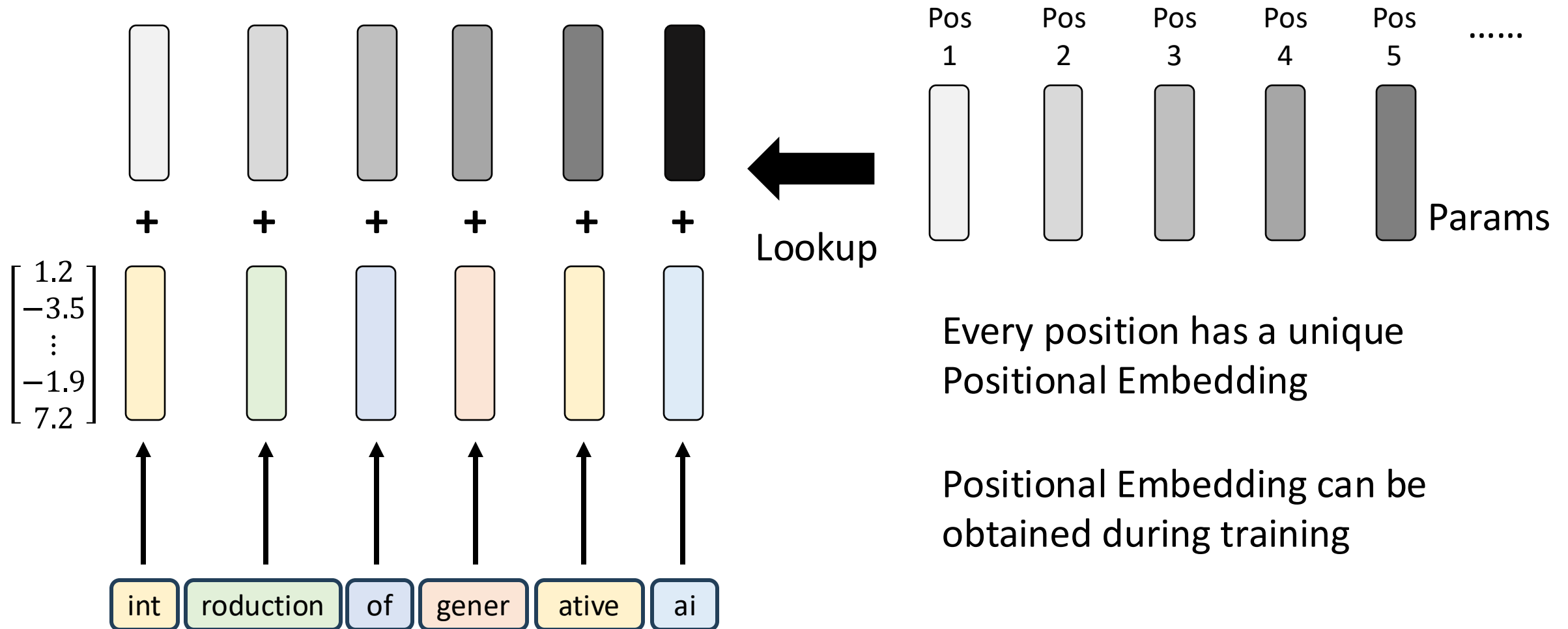
$$\begin{bmatrix} 1.2 \\ -3.5 \\ \vdots \\ -1.9 \\ 7.2 \end{bmatrix}$$


Step 2: Understand each token

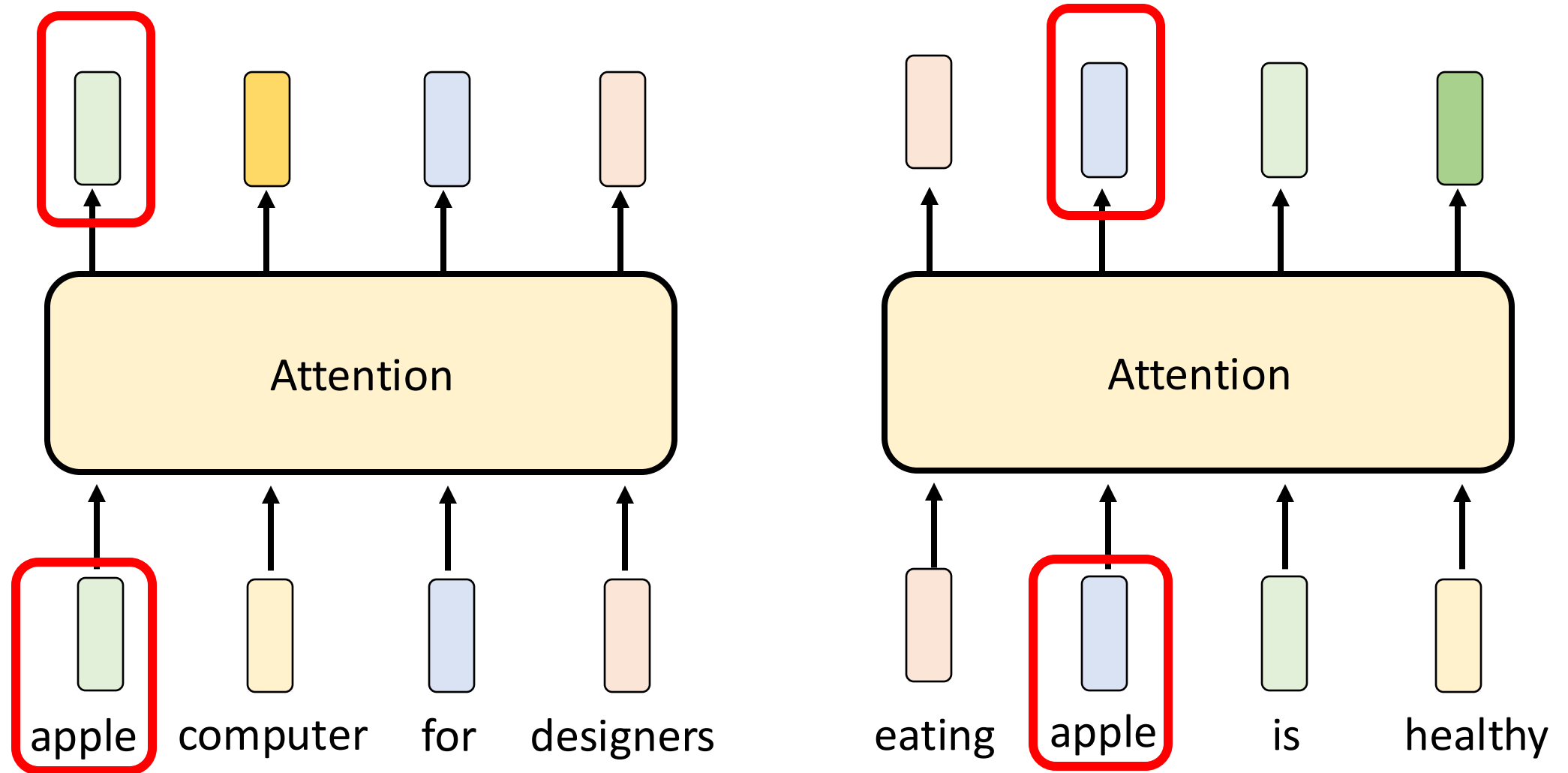
Tokens with similar meanings have similar embeddings.



Step 2: Understand each token - Position



Step 3: Attention: understand context



Tokens exchange information with each other

Step 3: Attention: understand context

Attention Is All You Need

<https://arxiv.org/abs/1706.03762>

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

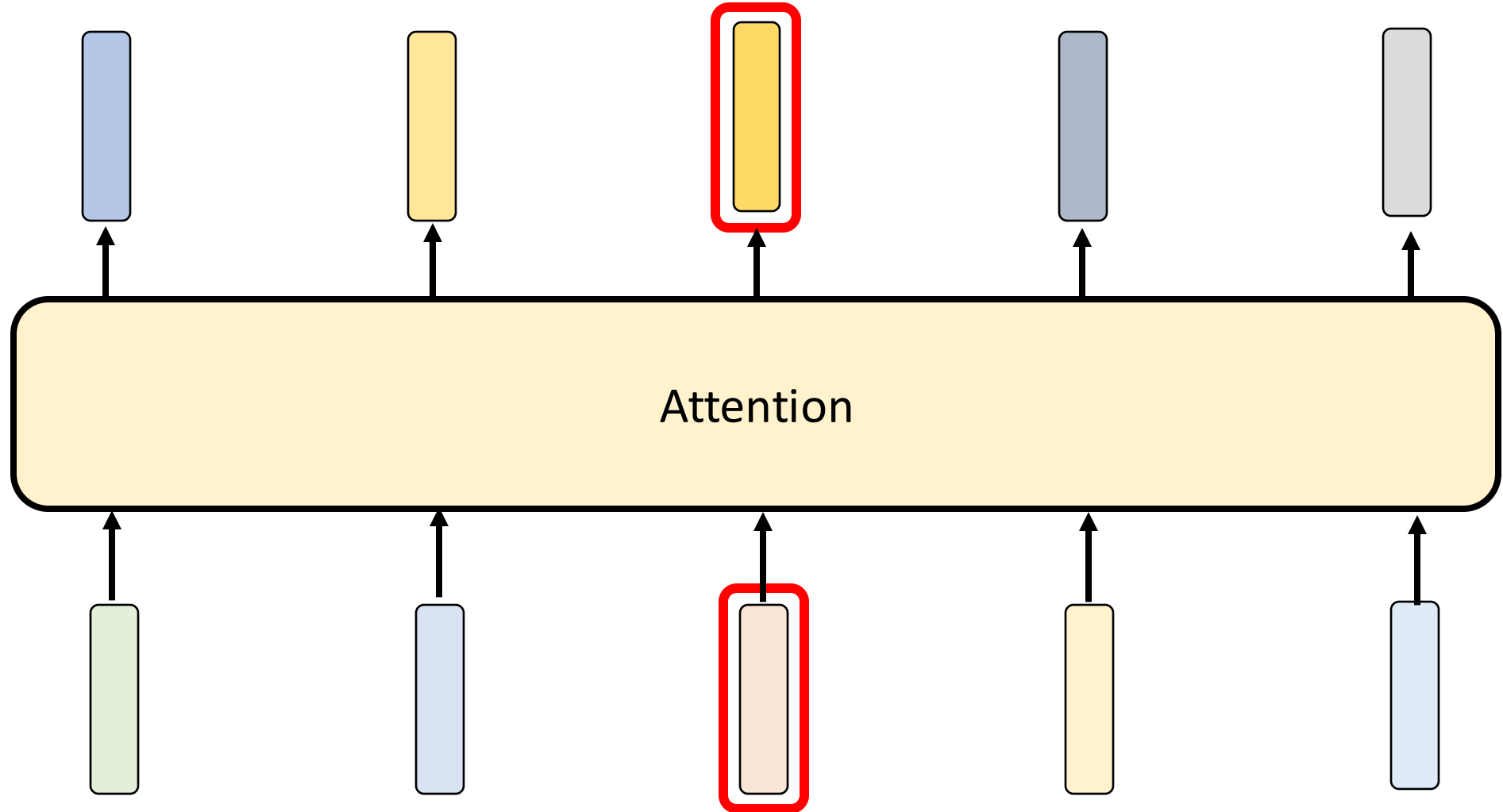
Aidan N. Gomez* †
University of Toronto
aidan@cs.toronto.edu

Łukasz Kaiser*
Google Brain
lukaszkaizer@google.com

Illia Polosukhin* ‡
illia.polosukhin@gmail.com

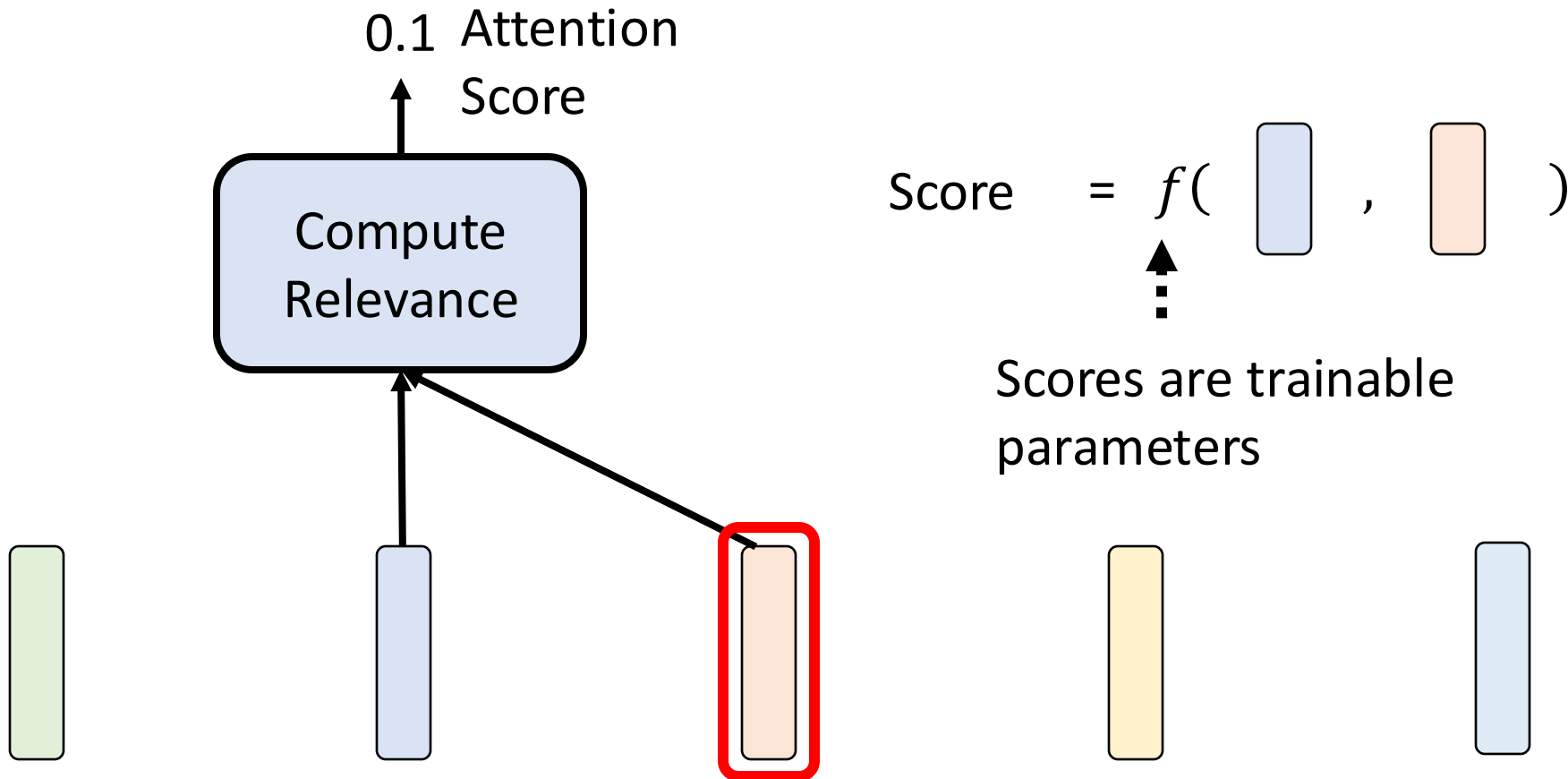
The key innovation of the Transformer paper: Recurrent Neural Networks (RNNs) are not required—Attention is sufficient.

Step 3: Attention: understand context



Step 3: Attention: understand context

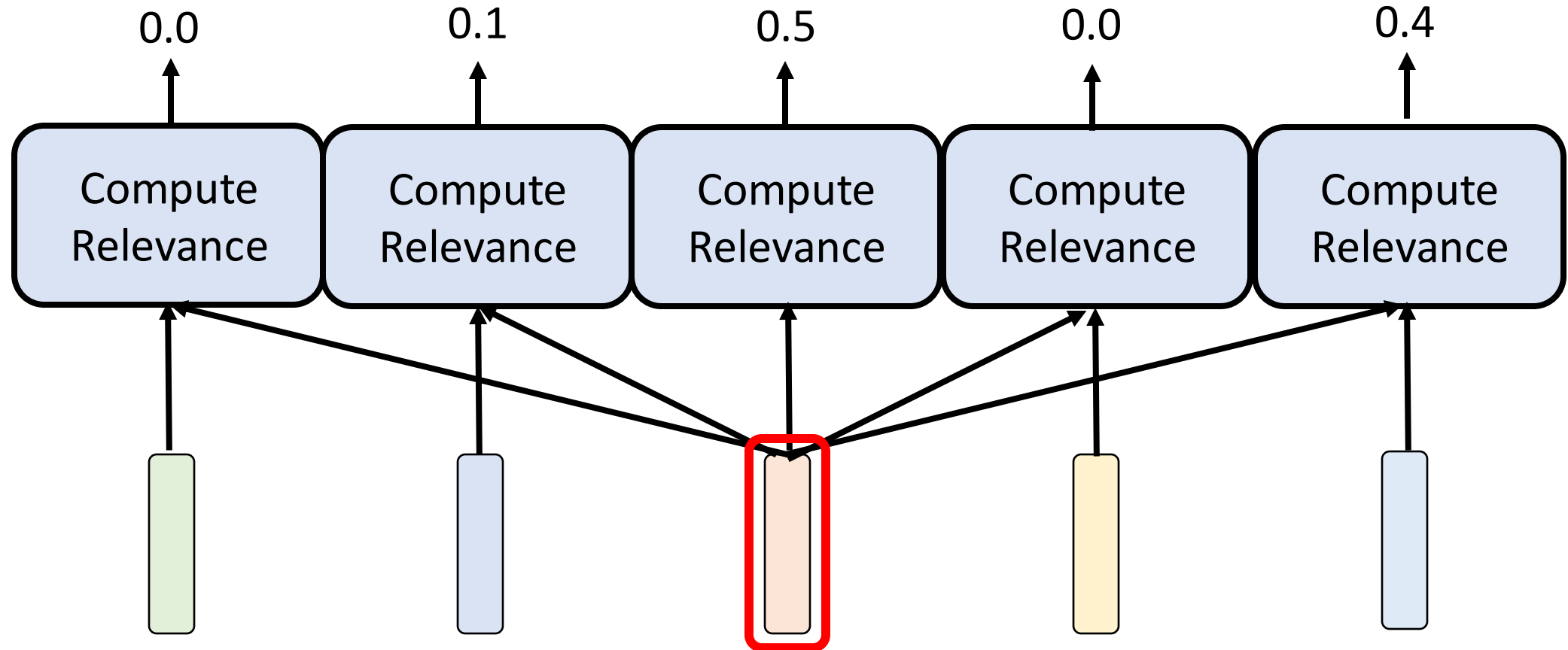
Finding relevant tokens



Step 3: Attention: understand context

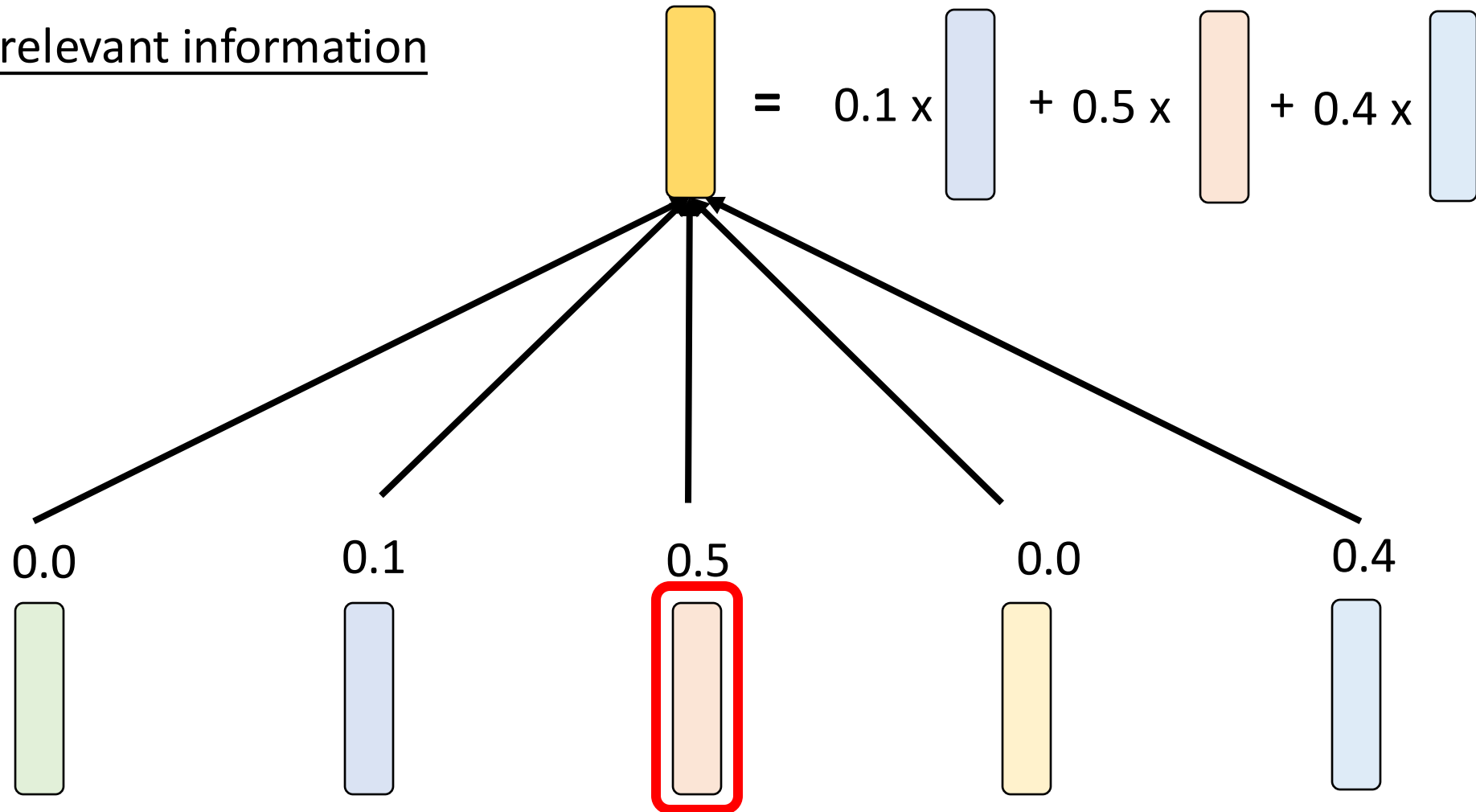
Finding relevant tokens

Attention Scores

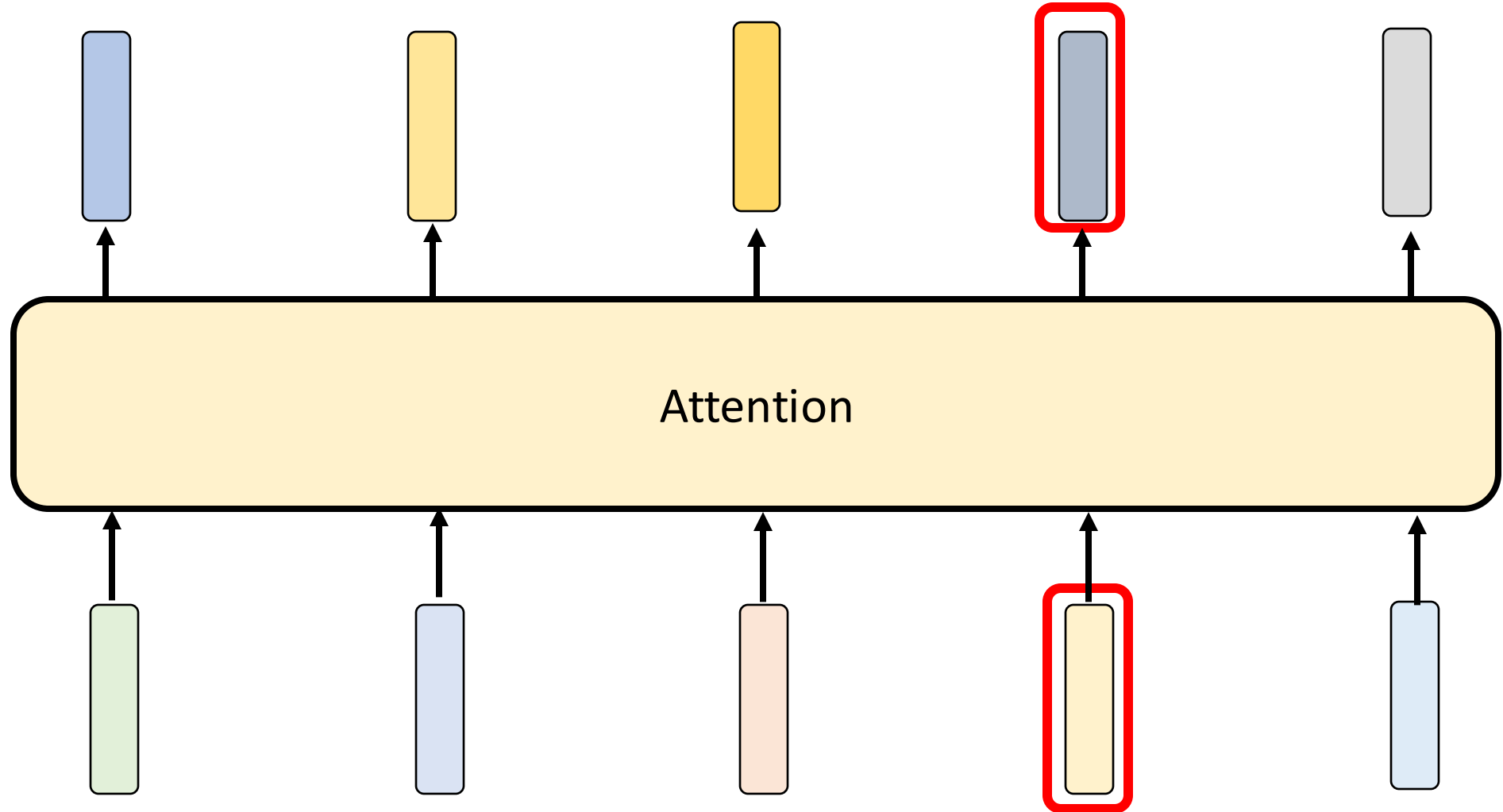


Step 3: Attention: understand context

Aggregate relevant information



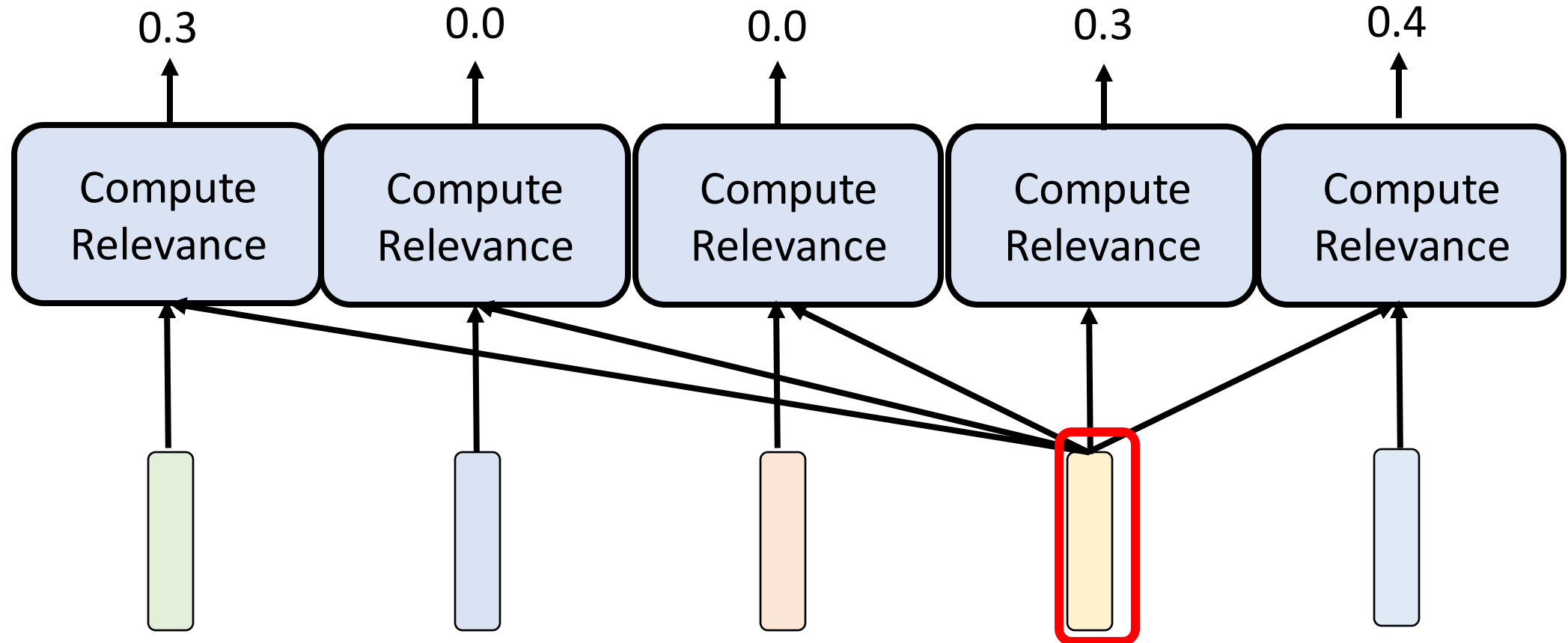
Step 3: Attention: understand context



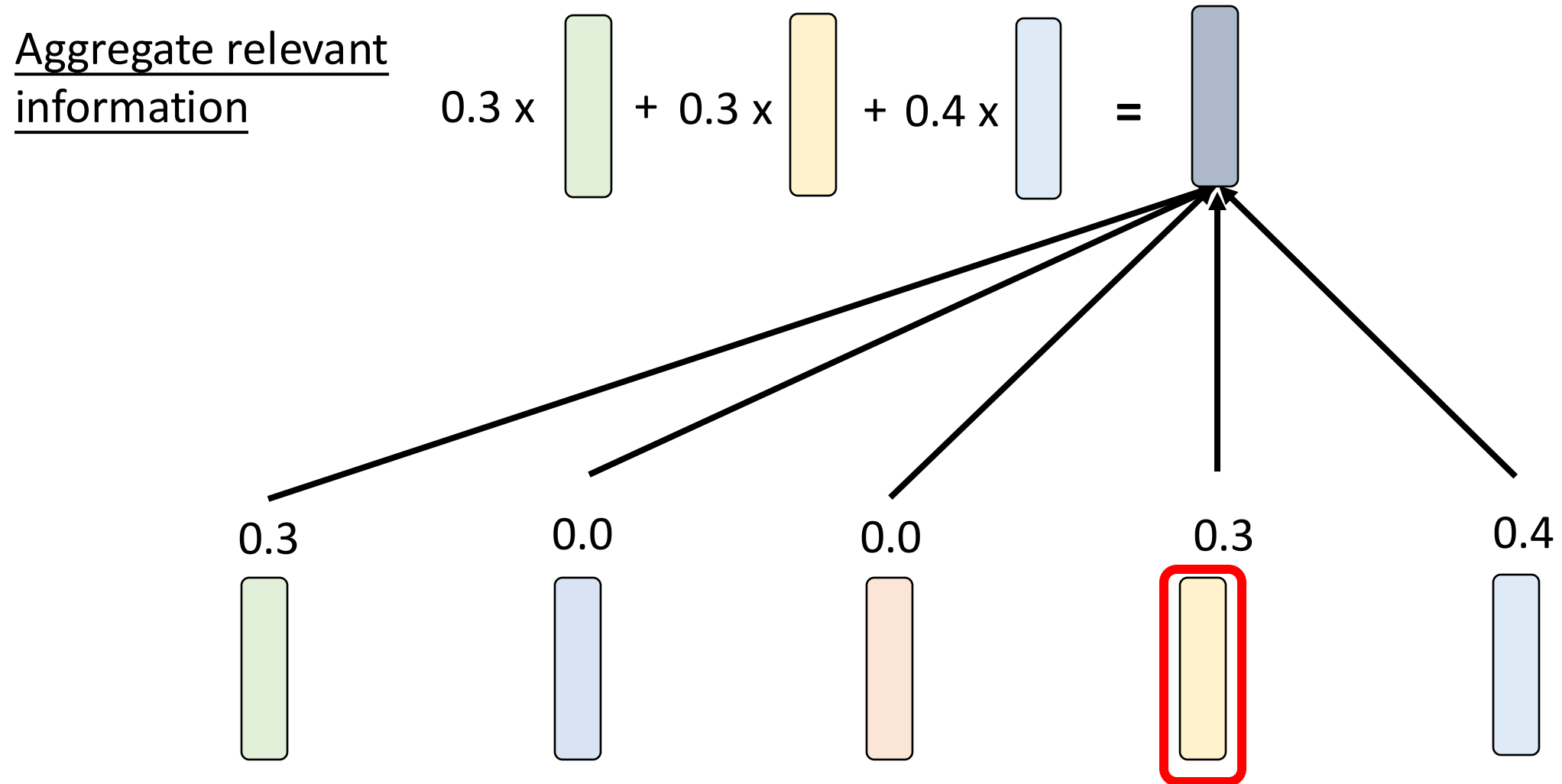
Repeat attention computations for next input token

Step 3: Attention: understand context

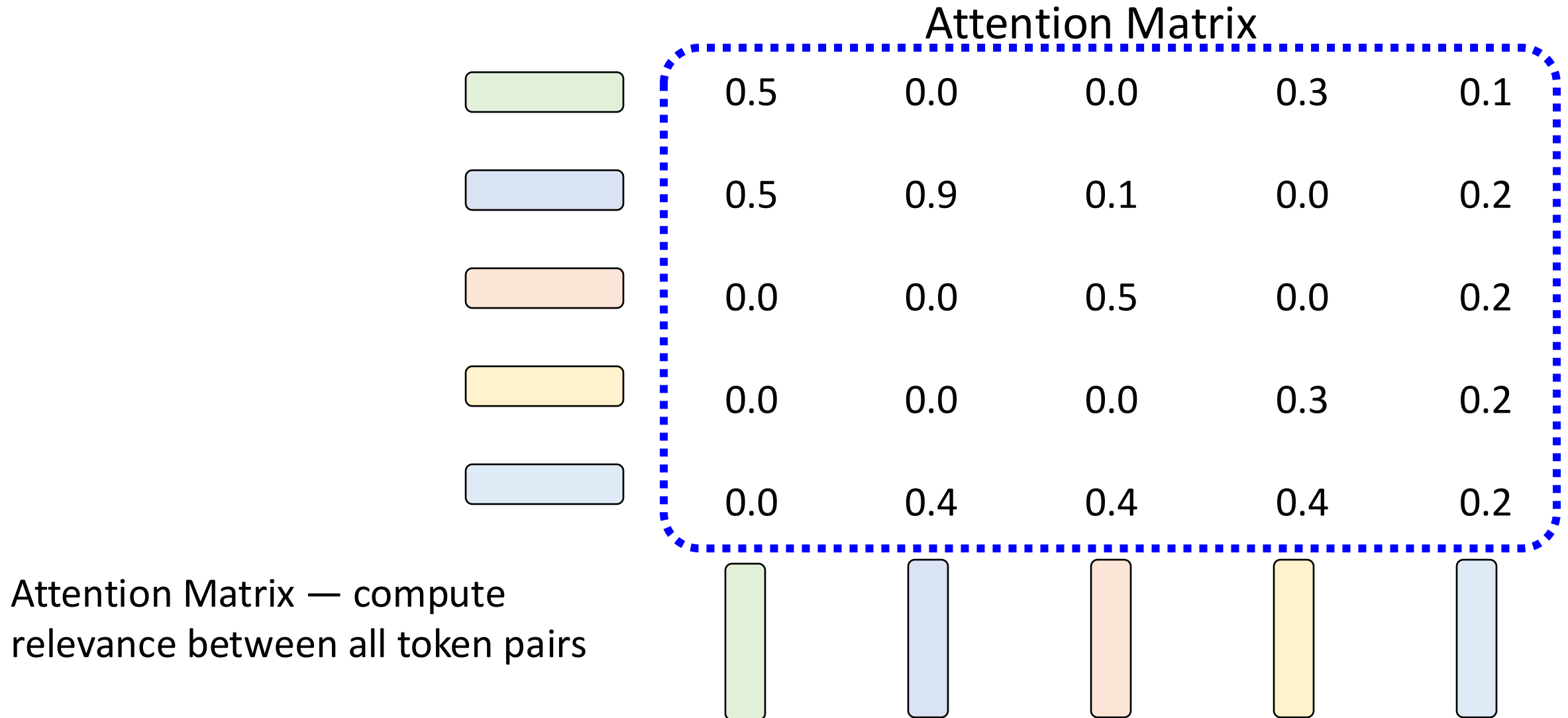
Finding relevant tokens



Step 3: Attention: understand context

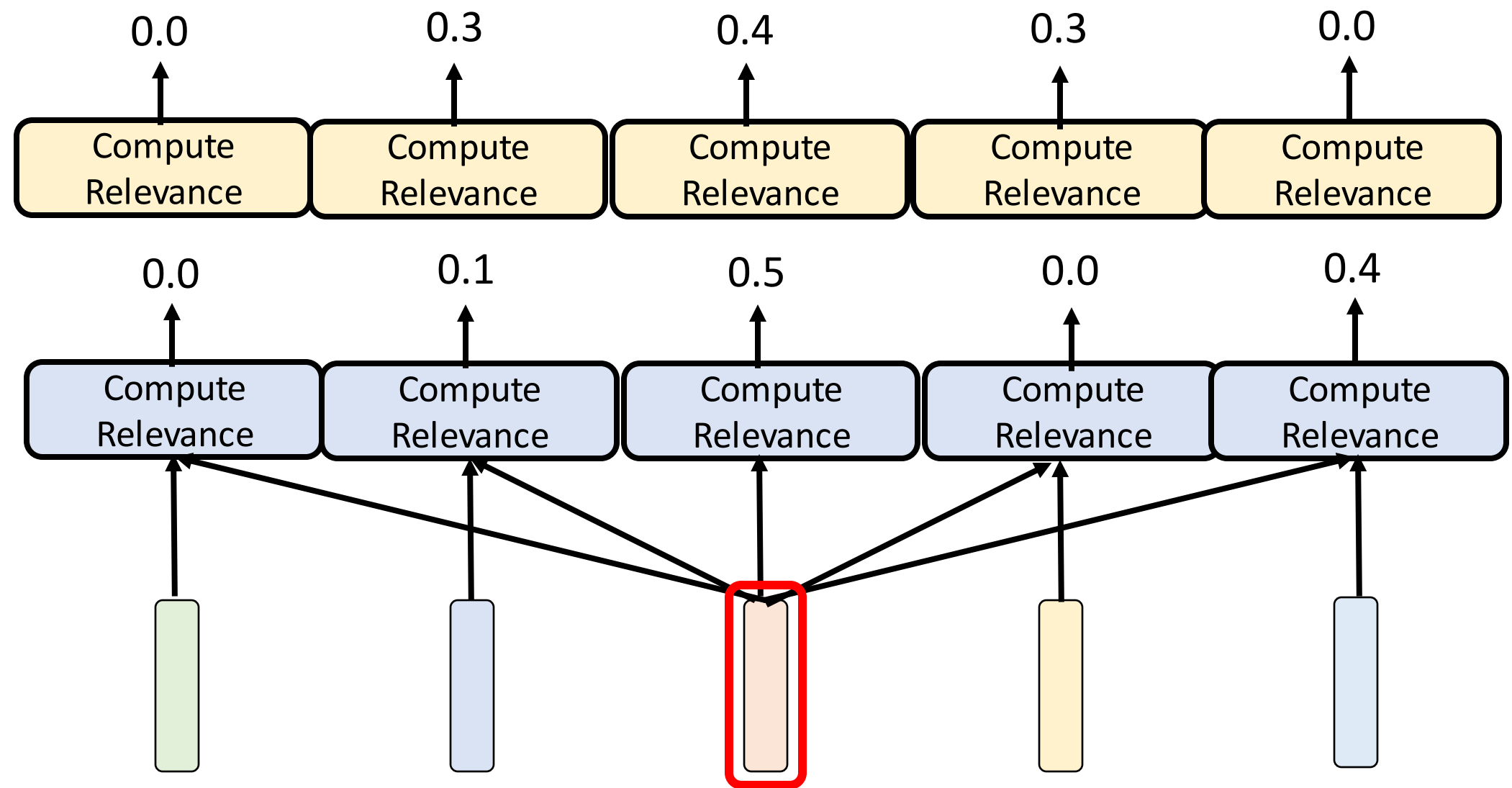


Step 3: Attention: understand context

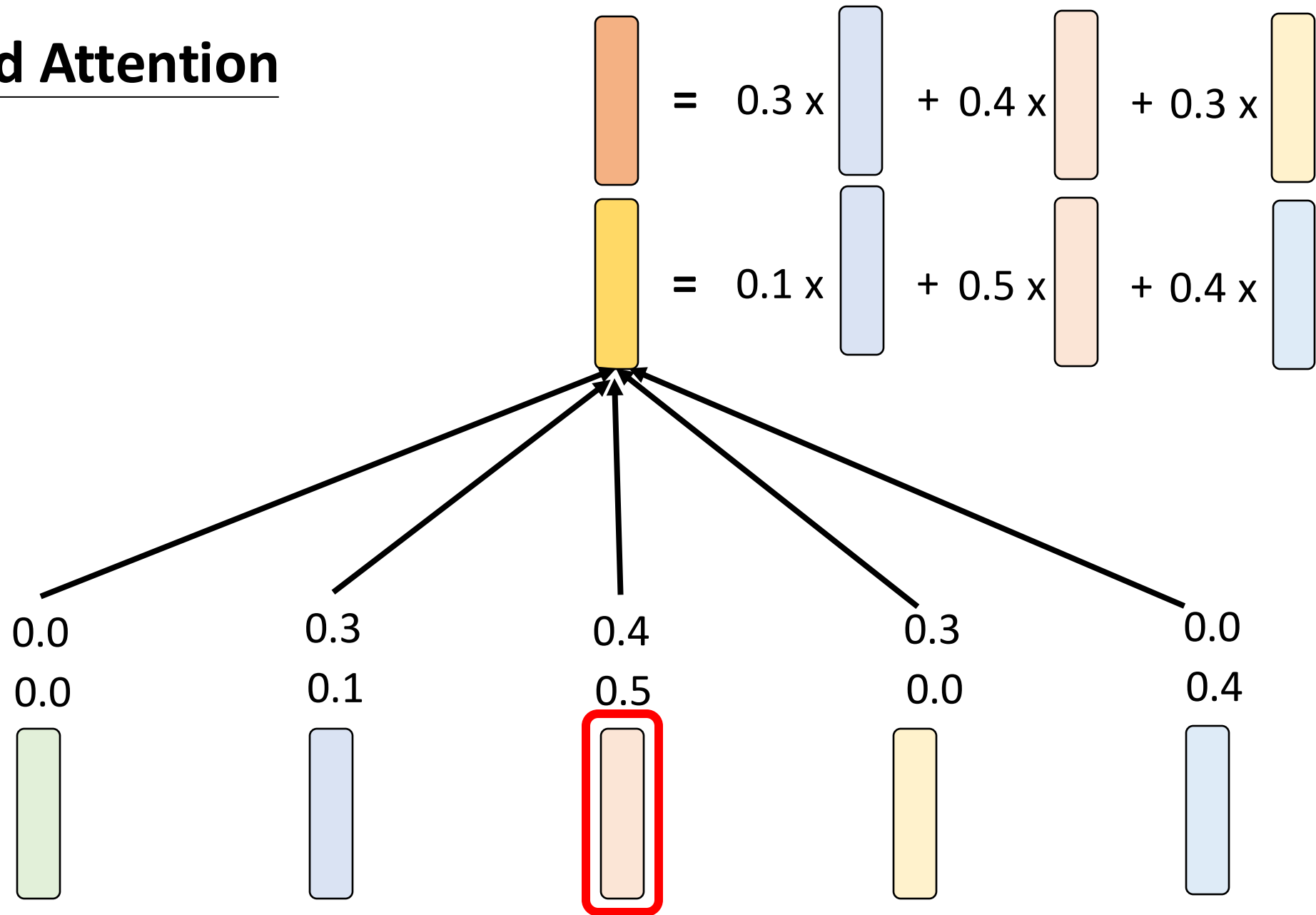


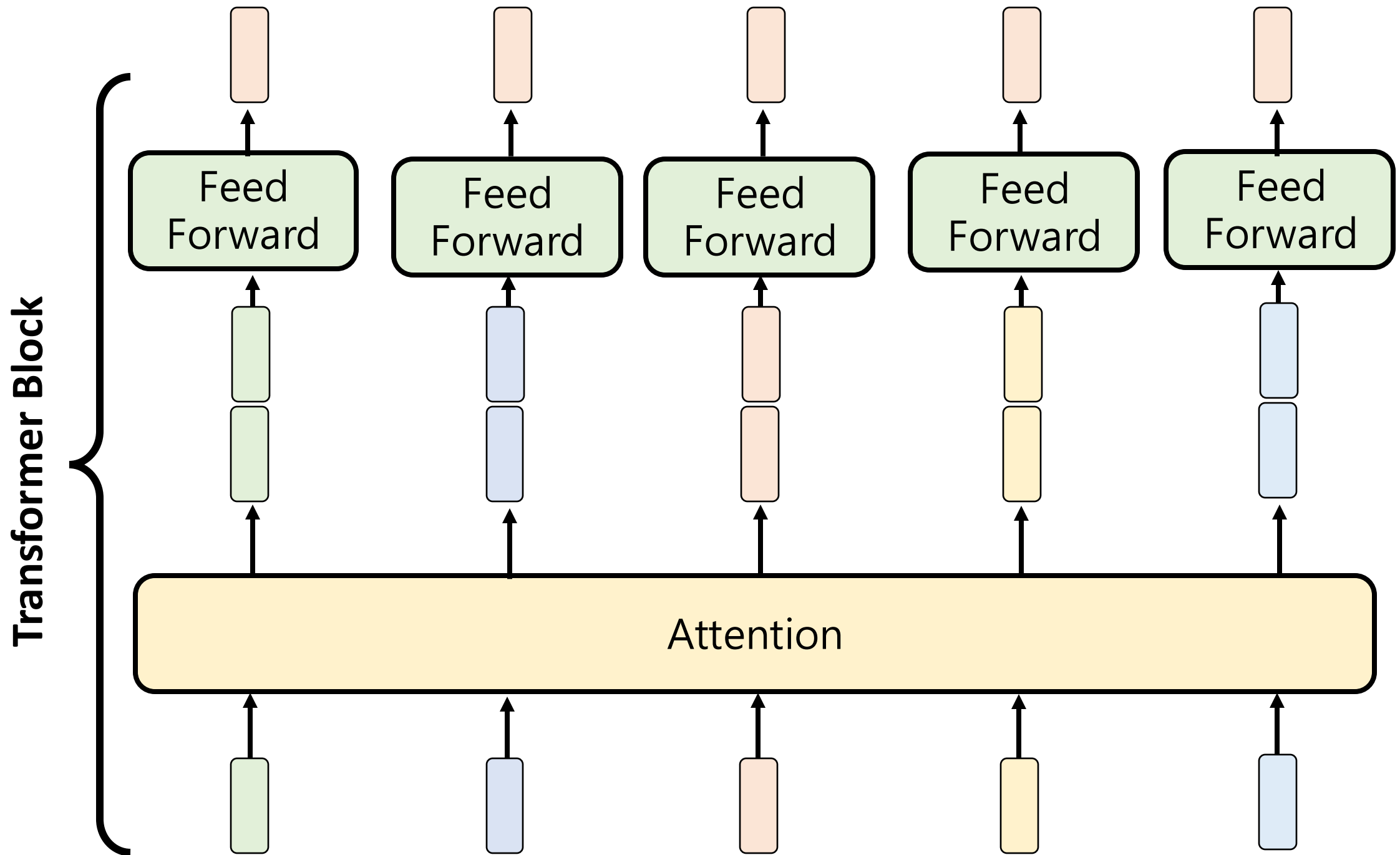
Multi-head Attention

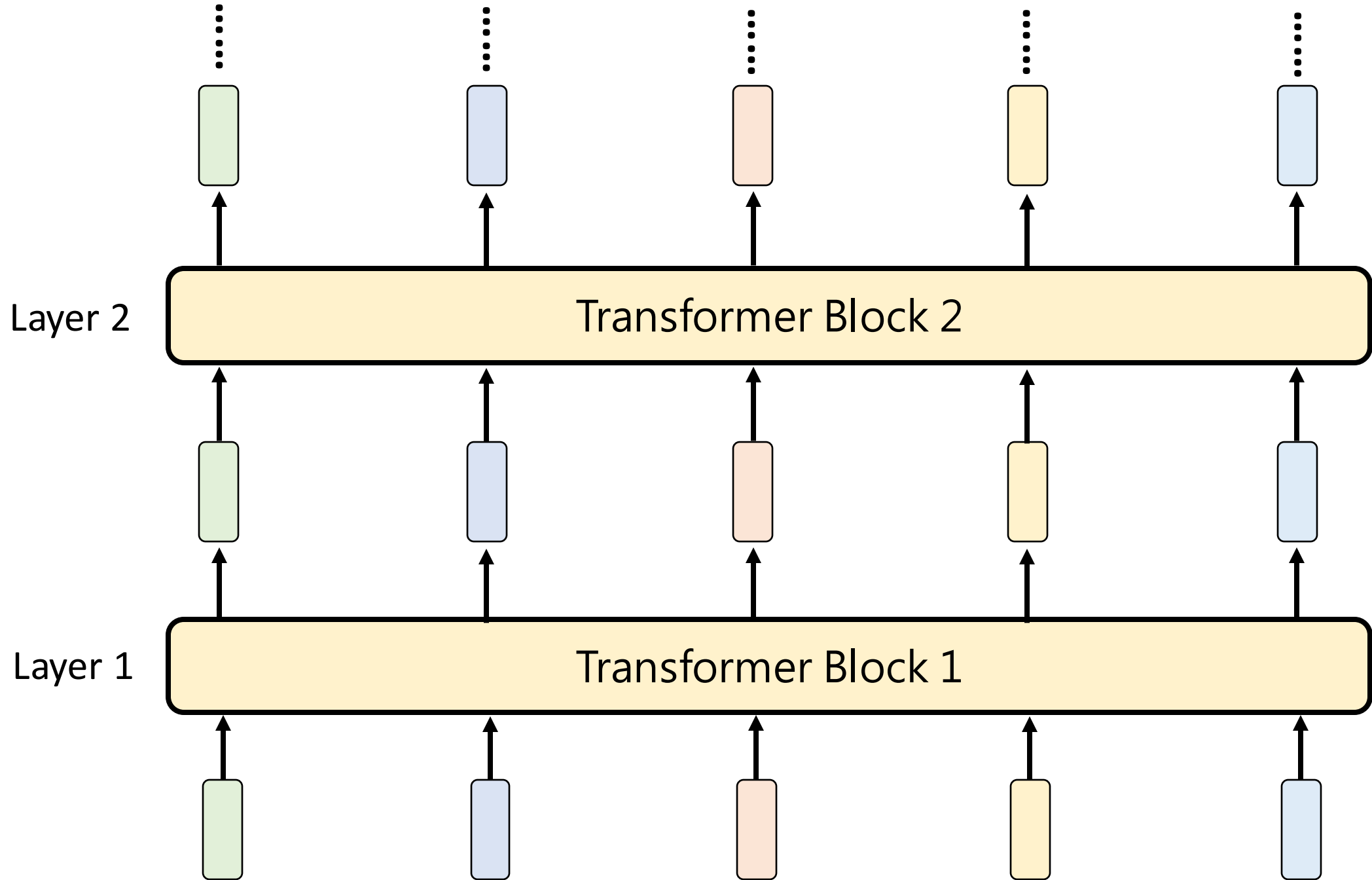
Multiple relevance types are calculated simultaneously for richer understanding

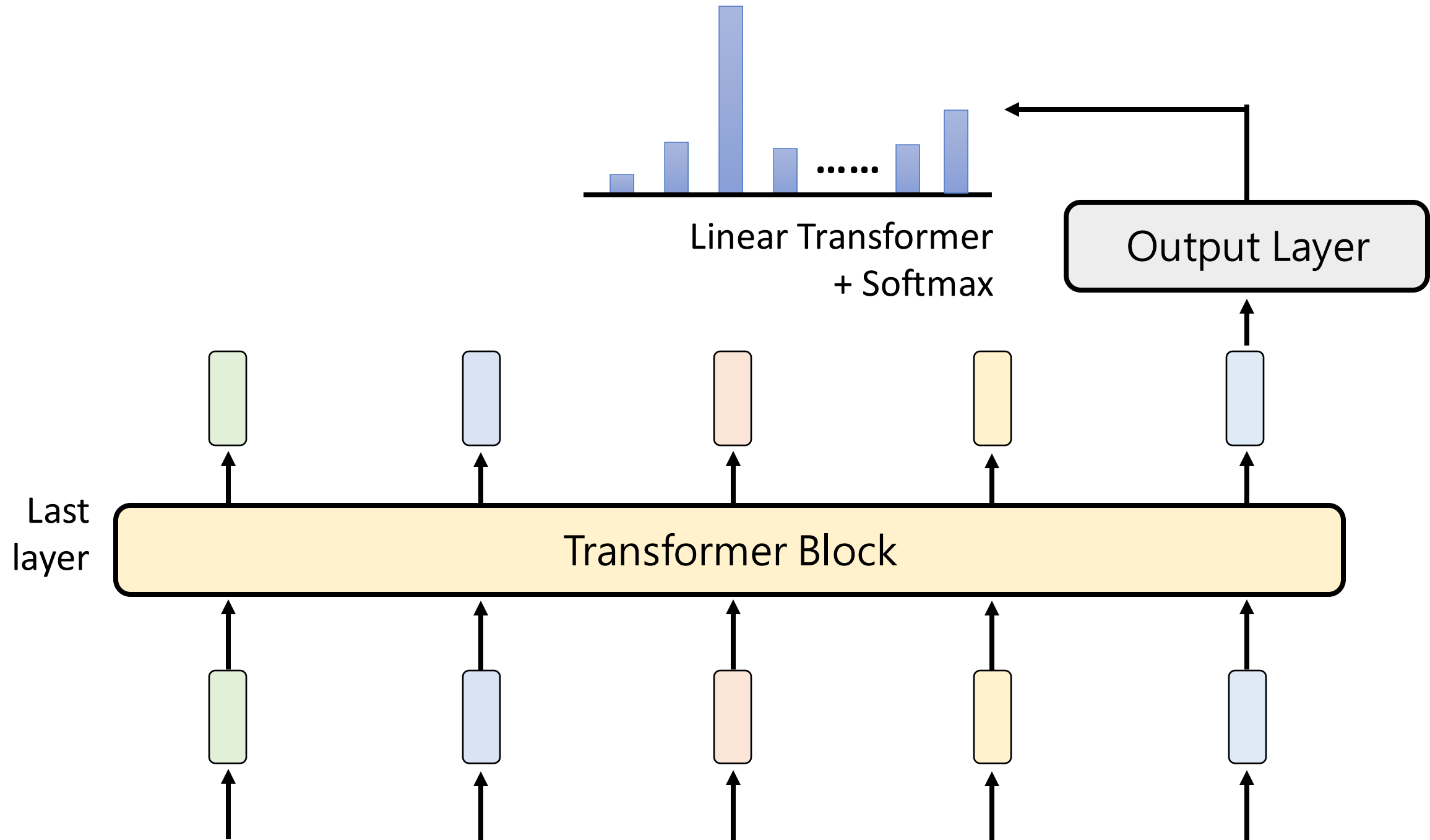


Multi-head Attention









Agenda

- Transformer Network
- **Naïve Bayes Classifier**
- K-means Clustering
- Boosting
- Model Generalization
- Evaluation Metrics

Bayes classification setting

- A probabilistic framework for solving classification problems
- X, Y are random variables: X represents data and Y represents label
- Bayes' Rule:

$$P(Y = y_k | X = x_i) = \frac{P(X = x_i | Y = y_k)P(Y = y_k)}{P(X = x_i)}$$

$P(Y = y_k | X = x_i)$ means given the data point x_i , the probability of a possible class y_k

- In detail

$$P(Y = y_k | X = x_i) = \frac{P(X_1 = x_{i,1} \wedge \cdots \wedge X_d = x_{i,d} | Y = y_k)P(Y = y_k)}{P(X_1 = x_{i,1} \wedge \cdots \wedge X_d = x_{i,d})}$$

The probability of a data point $P(X = x_i)$ is the joint probability of all features

Naïve Bayesian classifiers

- Idea: use the training data to estimate $P(X|Y)$ and $P(Y)$
- Then use Bayes rule to infer $P(Y|X_{new})$ for new data
- Example: How to classify the new record $X = (\text{'Yes'}, \text{'Single'}, 80K)$

Tid	Refund	Marital Status	Taxable Income	Evade
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

Find the class with the highest probability given the attribute values.

Maximum A Posteriori (MAP) estimate:

- Find the value y for class Y that maximizes $P(Y=y | X)$

How do we estimate $P(Y|X)$ for the different values of Y ?

- We want to estimate $P(Y=\text{Yes} | X)$ and $P(Y=\text{No} | X)$

Using Bayes' Rule for classification

- Compute posterior probability $P(Y \mid X_1, X_2, \dots, X_d)$ using the Bayes Rule

$$P(Y = y_k \mid X = x_i) = \frac{\overbrace{P(X_1 = x_{i,1} \wedge \dots \wedge X_d = x_{i,d} \mid Y = y_k)}^{\text{Necessary but impractical}} \overbrace{P(Y = y_k)}^{\text{Easy to estimate from data}}}{\underbrace{P(X_1 = x_{i,1} \wedge \dots \wedge X_d = x_{i,d})}_{\text{Unnecessary}}}$$

- **Maximum A Posteriori (MAP):** Choose Y that maximizes

$$P(Y \mid X_1, X_2, \dots, X_d)$$

- How to estimate class-conditional probability $P(X_1, X_2, \dots, X_d \mid Y)$?
 - Estimating the joint probability distribution is not practical

Conditional independence

- If we make assumption that the attributes are independent given the class label, estimation becomes feasible

$$P(X_1, X_2, \dots, X_d | Y) = \prod_{j=1}^d P(X_j | Y)$$

- X_1 and X_2 are conditionally independent given Y
 - if $P(X_1 | X_2, Y) = P(X_1 | Y)$
 - Example: Arm length X_1 and reading skills X_2
 - Young child has shorter arm length and limited reading skills, compared to adults
 - If age Y is fixed, no apparent relationship between arm length and reading skills
 - Arm length and reading skills are conditionally independent given age

Naïve Bayes classifier

- Assume conditional independence among attributes X_i when class Y_j is given:
 - $P(X_1, X_2, \dots, X_d | Y_j) = P(X_1 | Y_j) P(X_2 | Y_j) \dots P(X_d | Y_j)$
 - Now we can estimate $P(X_i | Y_j)$ for all X_i and Y_j combinations from the training data
 - New point $X = (X_1, \dots, X_n)$ is classified to class y if
$$P(Y = y | X) = P(Y = y) \prod_i P(X_i | y)$$
is maximum over all possible values of Y .

Example

Record

- $X = (\text{Refund} = \text{Yes}, \text{Status} = \text{Single}, \text{Income} = 80\text{K})$
- For the class $Y = \text{'Evade'}$, we want to compute:
 $P(Y = \text{Yes}|X)$ and $P(Y = \text{No}|X)$
- We compute:
 - $P(Y = \text{Yes}|X) = P(Y = \text{Yes}) * P(\text{Refund} = \text{Yes} | Y = \text{Yes})$
 $* P(\text{Status} = \text{Single} | Y = \text{Yes})$
 $* P(\text{Income} = 80\text{K} | Y = \text{Yes})$
 - $P(Y = \text{No}|X) = P(Y = \text{No}) * P(\text{Refund} = \text{Yes} | Y = \text{No})$
 $* P(\text{Status} = \text{Single} | Y = \text{No})$
 $* P(\text{Income} = 80\text{K} | Y = \text{No})$

How to estimate probabilities from data?

<i>Tid</i>	Refund	Marital Status	Taxable Income	Evade
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

Class Prior Probability:

$$P(Y = y) = \frac{N_y}{N}$$

N_y : Number of records with class y

N : Number of records

$$P(Y = \text{Yes}) = 3/10$$

$$P(Y = \text{No}) = 7/10$$

How to estimate probabilities from data?

<i>Tid</i>	Refund	Marital Status	Taxable Income	Evade
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

Discrete attributes:

$$P(X_i = x|Y = y) = \frac{N_{x,y}}{N_y}$$

$N_{x,y}$: number of instances having attribute $X_i = x$ and belong to class y

N_y : number of instances of class y

How to estimate probabilities from data?

<i>Tid</i>	Refund	Marital Status	Taxable Income	Evade
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

Discrete attributes:

$$P(X_i = x|Y = y) = \frac{N_{x,y}}{N_y}$$

$N_{x,y}$: number of instances having attribute $X_i = x$ and belong to class y

N_y : number of instances of class y

$$P(\text{Refund} = \text{Yes}|\text{No}) = 3/7$$

How to estimate probabilities from data?

<i>Tid</i>	Refund	Marital Status	Taxable Income	Evade
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

Discrete attributes:

$$P(X_i = x|Y = y) = \frac{N_{x,y}}{N_y}$$

$N_{x,y}$: number of instances having attribute $X_i = x$ and belong to class y

N_y : number of instances of class y

$$P(\text{Refund} = \text{Yes}|\text{Yes}) = 0$$

How to estimate probabilities from data?

<i>Tid</i>	Refund	Marital Status	Taxable Income	Evade
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

Discrete attributes:

$$P(X_i = x|Y = y) = \frac{N_{x,y}}{N_y}$$

$N_{x,y}$: number of instances having attribute $X_i = x$ and belong to class y

N_y : number of instances of class y

$$P(\text{Status} = \text{Single}|\text{No}) = 2/7$$

How to estimate probabilities from data?

<i>Tid</i>	Refund	Marital Status	Taxable Income	Evade
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

Discrete attributes:

$$P(X_i = x|Y = y) = \frac{N_{x,y}}{N_y}$$

$N_{x,y}$: number of instances having attribute $X_i = x$ and belong to class y

N_y : number of instances of class y

$$P(\text{Status} = \text{Single}|\text{Yes}) = 2/3$$

How to estimate probabilities from data?

- For continuous attributes:
 - **Discretization:** Partition the range into bins:
 - Replace continuous value with bin value
 - Attribute changed from continuous to ordinal
 - **Probability density estimation:**
 - Assume attribute follows a distribution (e.g., normal distribution)
 - Use data to estimate parameters of distribution through MLE (e.g., mean and standard deviation)
 - The MLE for mean and variance is computed by $\mu = \frac{1}{n} \sum_{i=1}^n x_i$ and $\sigma^2 = \frac{1}{n} \sum_{i=1}^n (x_i - \mu)^2$
 - Once probability distribution is known, use it to estimate the conditional probability $P(X_i|Y)$

MLE for Normal distribution:

<https://towardsdatascience.com/maximum-likelihood-estimation-explained-normal-distribution-6207b322e47f>

How to estimate probabilities from data?

<i>Tid</i>	Refund	Marital Status	Taxable Income	Evade
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

- Normal distribution:

$$P(X_i = x | Y_j) = \frac{1}{\sqrt{2\pi\sigma_{ij}^2}} e^{-\frac{(a-\mu_{ij})^2}{2\sigma_{ij}^2}}$$

- One for each (X_i, Y_j) pair
- For **Class=No**
 - sample mean $\mu = 110$
 - sample variance $\sigma^2 = 2975$
- For **Income = 80**

$$P(\text{Income} = 80 | \text{No}) = \frac{1}{\sqrt{2\pi 2975}} e^{-\frac{(80-110)^2}{2(2975)}} = 0.0062$$

How to estimate probabilities from data?

<i>Tid</i>	Refund	Marital Status	Taxable Income	Evade
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

- Normal distribution:

$$P(X_i = x | Y_j) = \frac{1}{\sqrt{2\pi\sigma_{ij}^2}} e^{-\frac{(x-\mu_{ij})^2}{2\sigma_{ij}^2}}$$

- One for each (X_i, Y_j) pair
- For **Class=Yes**
 - sample mean $\mu = 90$
 - sample variance $\sigma^2 = 25$
- For **Income = 80**

$$P(\text{Income} = 80 | \text{Yes}) = \frac{1}{\sqrt{2\pi 25}} e^{-\frac{(80-90)^2}{2(25)}} = 0.0108$$

Example

Record

- $X = (\text{Refund} = \text{Yes}, \text{Status} = \text{Single}, \text{Income} = 80\text{K})$
- For the class $Y = \text{'Evade'}$, we want to compute:
 $P(Y = \text{Yes}|X)$ and $P(Y = \text{No}|X)$
- We compute:
 - $P(Y = \text{Yes}|X) = P(Y = \text{Yes}) * P(\text{Refund} = \text{Yes} | Y = \text{Yes})$
 $\quad * P(\text{Status} = \text{Single} | Y = \text{Yes})$
 $\quad * P(\text{Income} = 80\text{K} | Y = \text{Yes})$
 $= 3/10 * 0 * 2/3 * 0.01 = 0$
 - $P(Y = \text{No}|X) = P(Y = \text{No}) * P(\text{Refund} = \text{Yes} | Y = \text{No})$
 $\quad * P(\text{Status} = \text{Single} | Y = \text{No})$
 $\quad * P(\text{Income} = 80\text{K} | Y = \text{No})$
 $= 7/10 * 3/7 * 2/7 * 0.0062 = 0.0005$

Laplace Smoothing

- If one of the conditional probabilities is **zero**, then the entire expression becomes zero
- Laplace Smoothing:

$$P(A_i = a|C = c) = \frac{N_{ac} + 1}{N_c + N_i}$$

- N_i : number of unique attribute **values** for attribute A_i

Example of Laplace Smoothing

- Creating a Naïve Bayes Classifier, essentially means to compute **counts**:

Total number of records: $N = 10$

Class No:

Number of records: 7

Attribute Refund:

Yes: 3

No: 4

Attribute Marital Status:

Single: 2

Divorced: 1

Married: 4

Attribute Income:

mean: 110

variance: 2975

Class Yes:

Number of records: 3

Attribute Refund:

Yes: 0

No: 3

Attribute Marital Status:

Single: 2

Divorced: 1

Married: 0

Attribute Income:

mean: 90

variance: 25

With Laplace Smoothing

naive Bayes Classifier:

$P(\text{Refund}=\text{Yes} | \text{No}) = 4/9$

$P(\text{Refund}=\text{No} | \text{No}) = 5/9$

$P(\text{Refund}=\text{Yes} | \text{Yes}) = 1/5 \quad (0+1)/(3+2)$

$P(\text{Refund}=\text{No} | \text{Yes}) = 4/5$

$P(\text{Marital Status}=\text{Single} | \text{No}) = 3/10$

$P(\text{Marital Status}=\text{Divorced} | \text{No}) = 2/10$

$P(\text{Marital Status}=\text{Married} | \text{No}) = 5/10$

$P(\text{Marital Status}=\text{Single} | \text{Yes}) = 3/6$

$P(\text{Marital Status}=\text{Divorced} | \text{Yes}) = 2/6$

$P(\text{Marital Status}=\text{Married} | \text{Yes}) = 1/6 \quad (0+1)/(3+3)$

For taxable income:

If class=No: sample mean=110
 sample variance=2975

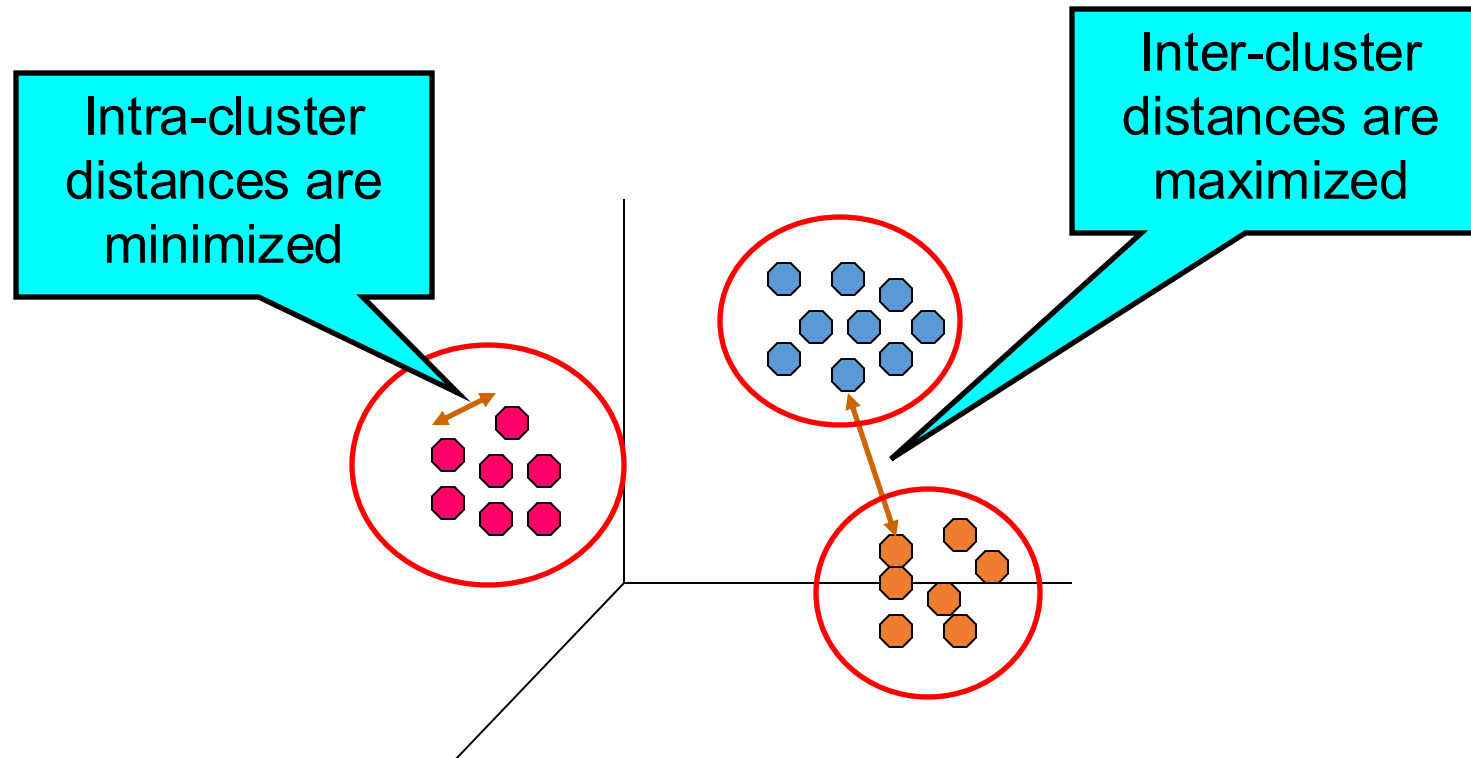
If class=Yes: sample mean=90
 sample variance=25

Agenda

- Transformer Network
- Naïve Bayes Classifier
- **K-means Clustering**
- Boosting
- Model Generalization
- Evaluation Metrics

What is clustering?

- Given a set of objects, **grouping** them such that
 - The objects in the same group (cluster) are similar (or related) to one another
 - The objects are different from (or unrelated to) the objects in other groups

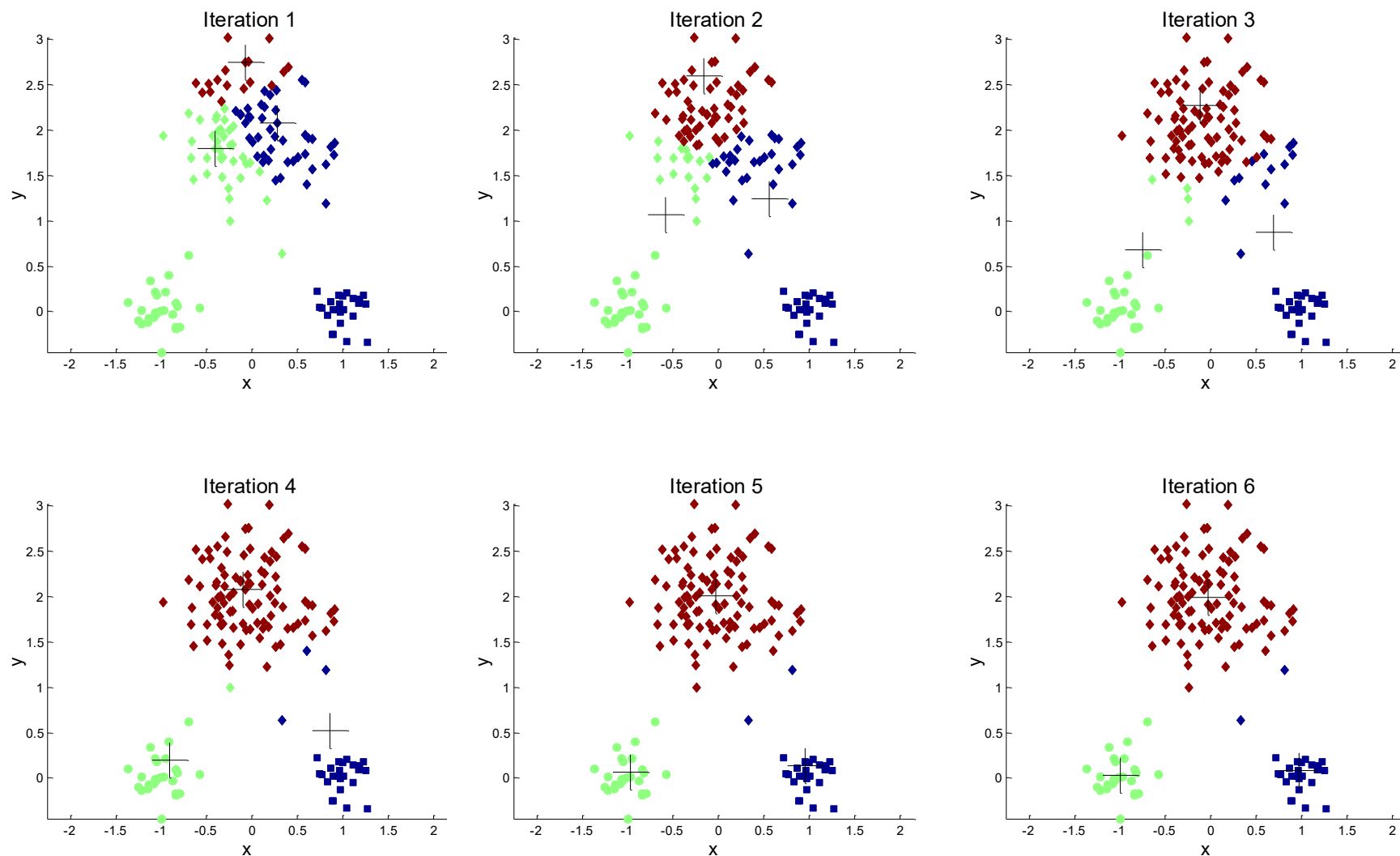


K-means algorithm

- Partitional clustering approach
- Each cluster is associated with a **centroid** (center point)
- Each point is assigned to the cluster with the **closest** centroid
- Number of clusters, **K**, must be specified
- Objective
 - **minimize the sum of distances** of all the points to their respective **centroid**

1. Select K points as the **initial centroids**
2. **repeat**
3. Form K clusters by assigning each point to the closest centroid
4. Compute the new **centroid*** of each cluster (**distance minimizing**)
5. **until** The centroids do not change

Example of k-means clustering



K-means algorithm details

- Simple iterative algorithm.
 - **Repeat** {assign each point to a nearest centroid; re-compute cluster centroids} until centroids stop changing.
- Initial centroids are often chosen randomly.
 - Clusters produced can **vary** from one run to another
- The centroid is (typically) the mean of the points in the cluster
 - **Mean** for SSE and cosine similarity
- Most of the convergence happens in the first few iterations.
 - Often the stopping condition is changed to “**until relatively few points change clusters**”
- Complexity is $O(n * K * I * d)$
 - n = number of points
 - K = number of clusters
 - I = number of iterations
 - d = number of attributes

Dealing with initialization

- Do **multiple runs** and select the clustering with the smallest error
- Select original set of points by methods other than random
 - E.g., pick the most distant (from each other) points as cluster centers (**K-means++** algorithm)

K-means++ algorithm

- Consistently produces better results in terms of SSE
 - By choosing the initial values (or "seeds") for the k-means clustering algorithm
- To make initialized centroids apart, perform the following

1. Select an initial point at random to be the first centroid
2. For $k - 1$ steps
 1. For each of the N points, x_i , $1 \leq i \leq N$, find the minimum squared distance to the existing centroids, $C_1, \dots, C_j$, $1 \leq j < k$, i.e., $\min_j d^2(C_j, x_i)$
 2. Randomly select a new centroid by choosing a point by probability $\frac{\min_j d^2(C_j, x_i)}{\sum_i \min_j d^2(C_j, x_i)}$
3. End For

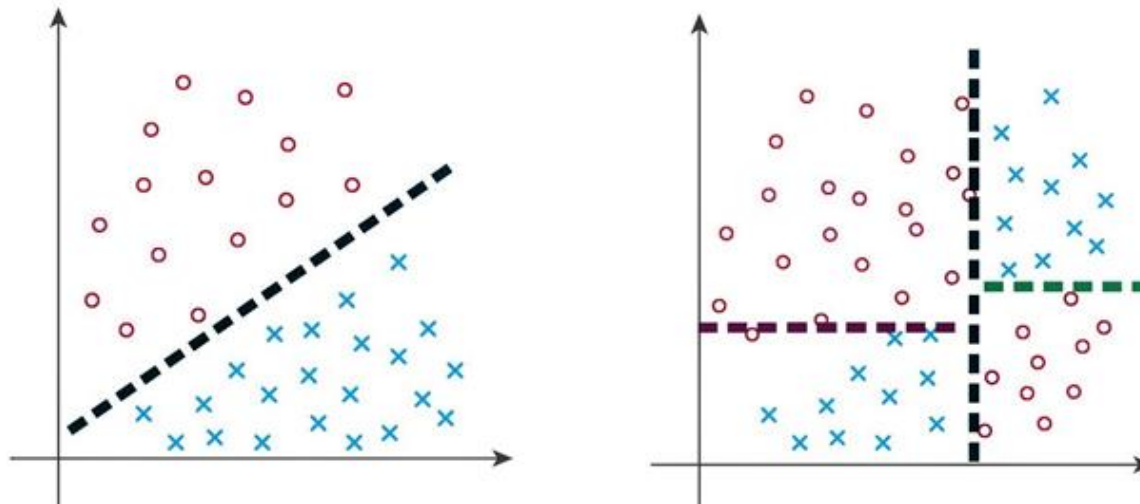
Make the new centroid far from the existing ones, and avoid choosing outliers!

Agenda

- Transformer Network
- Naïve Bayes Classifier
- K-means Clustering
- **Boosting**
- Model Generalization
- Evaluation Metrics

Weak learners

- Simple classifier that do only slightly better than random chance (50%)
- Like naïve Bayes, logistic regression, shallow decision tree

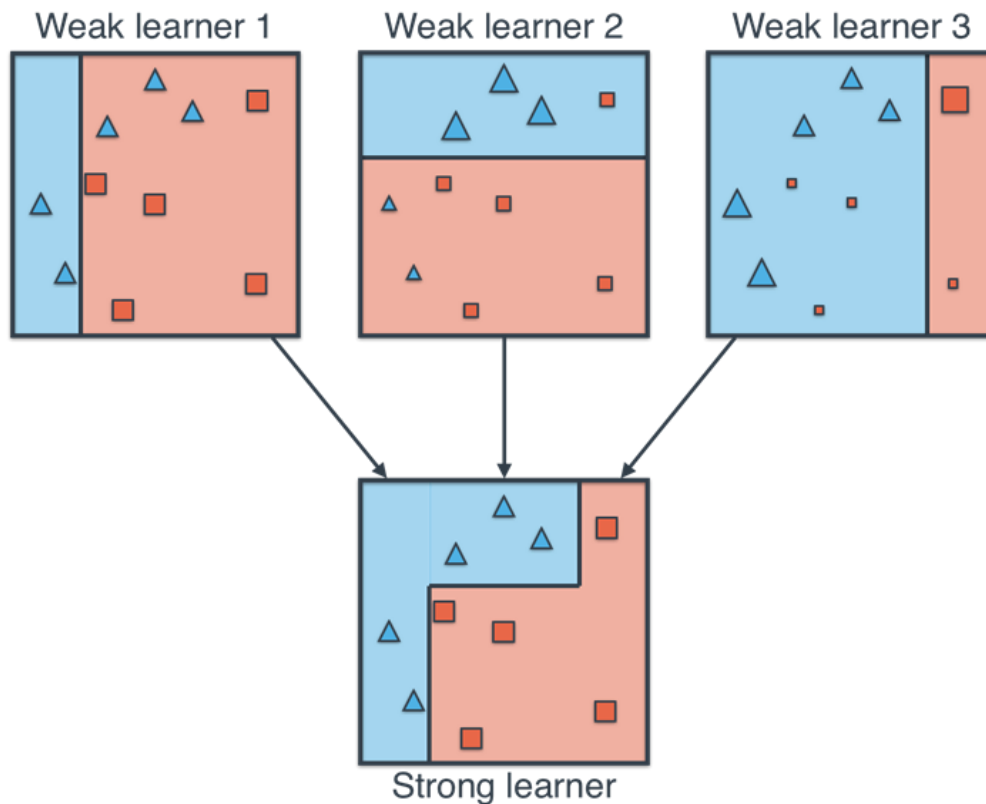


Good – Low variance, don't usually overfit (Simple “rule of thumb” are often correct)

Bad – High bias, can't solve hard learning problems (Can't find a single highly accurate prediction rule)

Ensemble methods

- Instead of learning a single (weak) classifier, learn **many weak classifiers that are good at different parts of the input space**
- Predict class label of test records by combining the predictions made by multiple classifiers (e.g., by taking majority vote)



We get a strong learner by assembling a set of weak learners – avengers of models!

AdaBoost explanation

INPUT: training data $X, y = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$,
the number of iterations T

1: Initialize a vector of n uniform weights $\mathbf{w}_1 = [\frac{1}{n}, \dots, \frac{1}{n}]$

2: **for** $t = 1, \dots, T$

3: Train model h_t on X, y with instance weights $\mathbf{w}_t$

4: Compute the weighted training error rate of h_t :

$$\epsilon_t = \sum_{i: y_i \neq h_t(\mathbf{x}_i)} w_{t,i}$$

5: Choose $\beta_t = \frac{1}{2} \ln \left(\frac{1-\epsilon_t}{\epsilon_t} \right)$

6: Update all instance weights:

$$w_{t+1,i} = w_{t,i} \exp(-\beta_t y_i h_t(\mathbf{x}_i)) \quad \forall i = 1, \dots, n$$

7: Normalize $\mathbf{w}_{t+1}$ to be a distribution:

$$w_{t+1,i} = \frac{w_{t+1,i}}{\sum_{j=1}^n w_{t+1,j}} \quad \forall i = 1, \dots, n$$

8: **end for**

9: **Return** the hypothesis

$$H(\mathbf{x}) = \text{sign} \left(\sum_{t=1}^T \beta_t h_t(\mathbf{x}) \right)$$

- $\mathbf{w}_t$ is a vector of weights over the training instances at iteration t
- All points start with equal weight

AdaBoost explanation

INPUT: training data $X, y = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$,
the number of iterations T

1: Initialize a vector of n uniform weights $\mathbf{w}_1 = [\frac{1}{n}, \dots, \frac{1}{n}]$

2: **for** $t = 1, \dots, T$

3: Train model h_t on X, y with instance weights $\mathbf{w}_t$

4: Compute the weighted training error rate of h_t :

$$\epsilon_t = \sum_{i: y_i \neq h_t(\mathbf{x}_i)} w_{t,i}$$

5: Choose $\beta_t = \frac{1}{2} \ln \left(\frac{1-\epsilon_t}{\epsilon_t} \right)$

6: Update all instance weights:

$$w_{t+1,i} = w_{t,i} \exp(-\beta_t y_i h_t(\mathbf{x}_i)) \quad \forall i = 1, \dots, n$$

7: Normalize $\mathbf{w}_{t+1}$ to be a distribution:

$$w_{t+1,i} = \frac{w_{t+1,i}}{\sum_{j=1}^n w_{t+1,j}} \quad \forall i = 1, \dots, n$$

8: **end for**

9: **Return** the hypothesis

$$H(\mathbf{x}) = \text{sign} \left(\sum_{t=1}^T \beta_t h_t(\mathbf{x}) \right)$$

- We need a way to weight instances differently when learning the model
- A simple way is to incorporate weights w into the cost function

$$J_{\text{reg}}(\boldsymbol{\theta}) = - \sum_{i=1}^n w_i [y_i \log h_{\boldsymbol{\theta}}(\mathbf{x}_i) + (1 - y_i) \log (1 - h_{\boldsymbol{\theta}}(\mathbf{x}_i))]$$

Weights of $\mathbf{x}_i$ Cross-entropy loss of $\mathbf{x}_i$

AdaBoost explanation

INPUT: training data $X, y = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$,
the number of iterations T

1: Initialize a vector of n uniform weights $\mathbf{w}_1 = [\frac{1}{n}, \dots, \frac{1}{n}]$

2: **for** $t = 1, \dots, T$

3: Train model h_t on X, y with instance weights $\mathbf{w}_t$

4: Compute the weighted training error rate of h_t :

$$\epsilon_t = \sum_{i: y_i \neq h_t(\mathbf{x}_i)} w_{t,i}$$

5: Choose $\beta_t = \frac{1}{2} \ln \left(\frac{1-\epsilon_t}{\epsilon_t} \right)$

6: Update all instance weights:

$$w_{t+1,i} = w_{t,i} \exp(-\beta_t y_i h_t(\mathbf{x}_i)) \quad \forall i = 1, \dots, n$$

7: Normalize $\mathbf{w}_{t+1}$ to be a distribution:

$$w_{t+1,i} = \frac{w_{t+1,i}}{\sum_{j=1}^n w_{t+1,j}} \quad \forall i = 1, \dots, n$$

8: **end for**

9: **Return** the hypothesis

$$H(\mathbf{x}) = \text{sign} \left(\sum_{t=1}^T \beta_t h_t(\mathbf{x}) \right)$$

- Error is the sum of the weights of all misclassified instances

- β_t measures the importance of h_t
- β_t grows as the error of h_t shrinks

AdaBoost explanation

INPUT: training data $X, y = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$,
the number of iterations T

1: Initialize a vector of n uniform weights $\mathbf{w}_1 = [\frac{1}{n}, \dots, \frac{1}{n}]$

2: **for** $t = 1, \dots, T$

3: Train model h_t on X, y with instance weights $\mathbf{w}_t$

4: Compute the weighted training error rate of h_t :

$$\epsilon_t = \sum_{i: y_i \neq h_t(\mathbf{x}_i)} w_{t,i}$$

5: Choose $\beta_t = \frac{1}{2} \ln \left(\frac{1-\epsilon_t}{\epsilon_t} \right)$

6: Update all instance weights:

$$w_{t+1,i} = w_{t,i} \exp(-\beta_t y_i h_t(\mathbf{x}_i)) \quad \forall i = 1, \dots, n$$

7: Normalize $\mathbf{w}_{t+1}$ to be a distribution:

$$w_{t+1,i} = \frac{w_{t+1,i}}{\sum_{j=1}^n w_{t+1,j}} \quad \forall i = 1, \dots, n$$

8: **end for**

9: **Return** the hypothesis

$$H(\mathbf{x}) = \text{sign} \left(\sum_{t=1}^T \beta_t h_t(\mathbf{x}) \right)$$

- Emphasize the misclassified instances by increase their weights

This is the same as:

$$w_{t+1,i} = w_{t,i} \times \begin{cases} e^{-\beta_t} & \text{if } h_t(\mathbf{x}_i) = y_i \\ e^{\beta_t} & \text{if } h_t(\mathbf{x}_i) \neq y_i \end{cases}$$

will be ≤ 1
will be ≥ 1

- Make the weight for next iteration $\mathbf{w}_{t+1}$ sum up to 1

AdaBoost explanation

INPUT: training data $X, y = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$,
the number of iterations T

1: Initialize a vector of n uniform weights $\mathbf{w}_1 = [\frac{1}{n}, \dots, \frac{1}{n}]$

2: **for** $t = 1, \dots, T$

3: Train model h_t on X, y with instance weights $\mathbf{w}_t$

4: Compute the weighted training error rate of h_t :

$$\epsilon_t = \sum_{i: y_i \neq h_t(\mathbf{x}_i)} w_{t,i}$$

5: Choose $\beta_t = \frac{1}{2} \ln \left(\frac{1-\epsilon_t}{\epsilon_t} \right)$

6: Update all instance weights:

$$w_{t+1,i} = w_{t,i} \exp(-\beta_t y_i h_t(\mathbf{x}_i)) \quad \forall i = 1, \dots, n$$

7: Normalize $\mathbf{w}_{t+1}$ to be a distribution:

$$w_{t+1,i} = \frac{w_{t+1,i}}{\sum_{j=1}^n w_{t+1,j}} \quad \forall i = 1, \dots, n$$

8: **end for**

9: **Return** the hypothesis

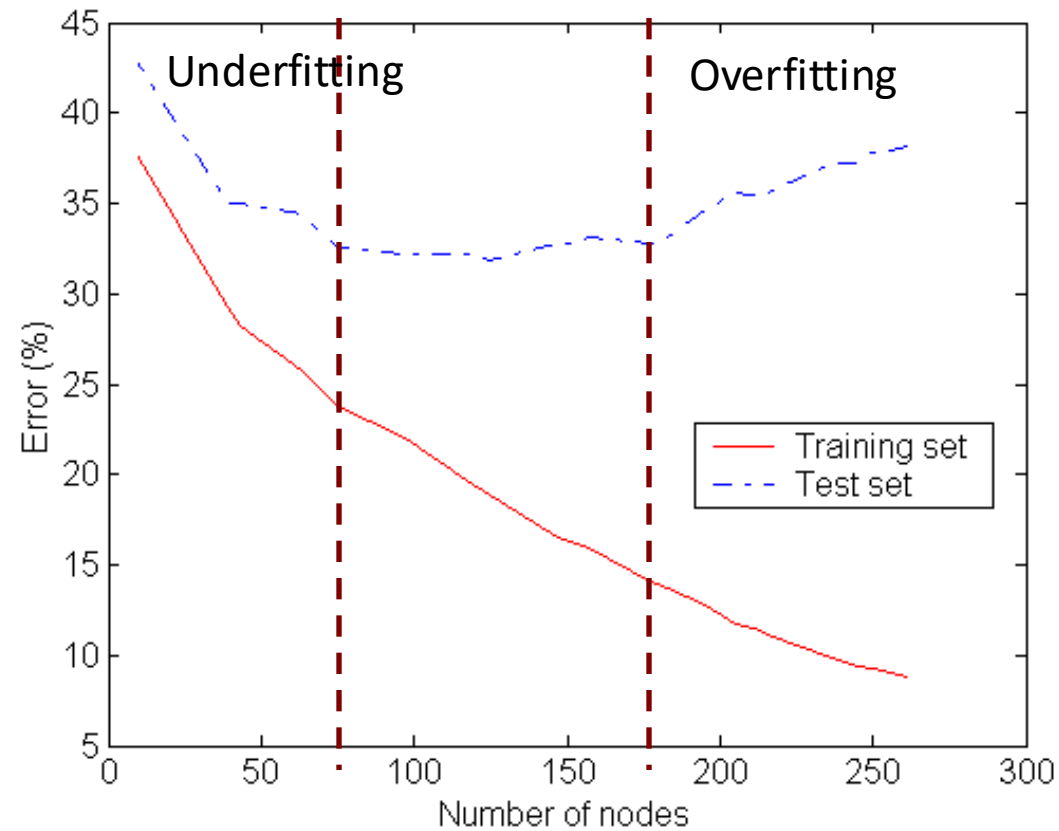
$$H(\mathbf{x}) = \text{sign} \left(\sum_{t=1}^T \beta_t h_t(\mathbf{x}) \right)$$

- Member classifiers with less error are given more weight in the final ensemble model
- Final prediction is a weighted combination of each member's prediction

Agenda

- Transformer Network
- Naïve Bayes Classifier
- K-means Clustering
- Boosting
- **Model Generalization**
- Evaluation Metrics

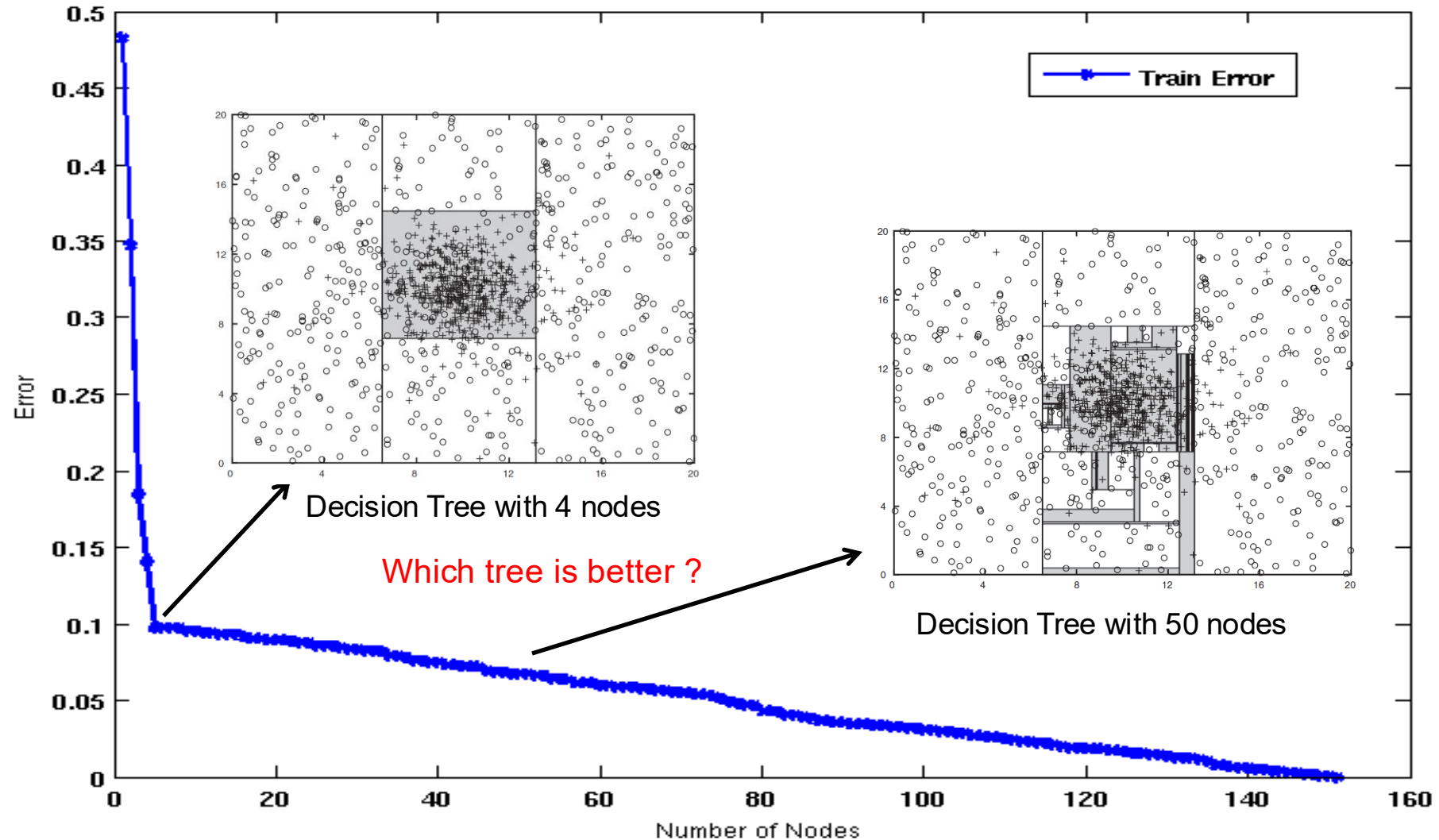
Underfitting and Overfitting



Underfitting: when model is **too simple**, both training and test errors are large

Overfitting: when model is **too complex** it models the details of the training set and fails on the test set

Which tree is better?



Notes on overfitting

- Overfitting results in decision trees that are **more complex** than necessary
- **Training error** no longer provides a good estimate of **test error**, that is, how well the tree will perform on previously unseen records
 - The model does not **generalize** well
- **Generalization**: The ability of the model to predict data points that it has not already seen.
- Need new ways for estimating **generalization errors**

Regularization for linear regression

- A method to control the complexity of model
- Why - address overfitting issues by keeping w small
- How - Penalize for large value of w_j
 - Can incorporate into the loss function
 - Works well when we have a lot of features

$$L(f) = \underbrace{\sum_{i=1}^n \left(y^{(i)} - f(x^{(i)}) \right)^2}_{\text{model fit to data}} + \lambda \underbrace{\sum_{j=1}^d w_j^2}_{\text{regularization}} \quad \text{Also called } L^2\text{-norm}$$

- λ is the predefined hyperparameter to control the degree of regularization
- No regularization on w_0 (bias b)

Agenda

- Transformer Network
- Naïve Bayes Classifier
- K-means Clustering
- Boosting
- Model Generalization
- **Evaluation Metrics**

Metrics for classification task

- Confusion Matrix (binary classification):

ACTUAL CLASS	PREDICTED CLASS	
	Class=Yes	Class=No
Class=Yes	a	b
	c	d

a: TP (true positive)

b: FN (false negative)

c: FP (false positive)

d: TN (true negative)

Metrics for performance evaluation

ACTUAL CLASS	PREDICTED CLASS	
	Class=Yes	Class=No
Class=Yes	a (TP)	b (FN)
	c (FP)	d (TN)

- Most widely-used metric:

$$\text{Accuracy} = \frac{a + d}{a + b + c + d} = \frac{TP + TN}{TP + TN + FP + FN}$$

Precision-Recall

$$\text{Precision (p)} = \frac{a}{a + c} = \frac{TP}{TP + FP}$$

$$\text{Recall (r)} = \frac{a}{a + b} = \frac{TP}{TP + FN}$$

$$\text{F-measure (F)} = \frac{1}{\left(\frac{1/r + 1/p}{2}\right)} = \frac{2rp}{r + p} = \frac{2a}{2a + b + c} = \frac{2TP}{2TP + FP + FN}$$

Count	PREDICTED CLASS		
ACTUAL CLASS		Class=Yes	Class=No
	Class=Yes	a	b
	Class=No	c	d

Assumption: The class YES is the one we care about.

- Precision is biased towards **C(Yes|Yes)** & **C(Yes|No)**
- Recall is biased towards **C(Yes|Yes)** & **C(No|Yes)**
- F-measure is biased towards all **except C(No|No)**

ROC (Receiver Operating Characteristic)

- **ROC** curve plots the trade-off between True Positive Rate (TPR) and False Positive Rate (FPR)
 - Changing **threshold** of the model to classify data (e.g., prob=0.5 to predict positive class)

Look at the **positive** predictions of the classifier and compute:

$$TPR = \frac{TP}{TP + FN}$$

What fraction of **positive instances** are predicted **correctly** ?

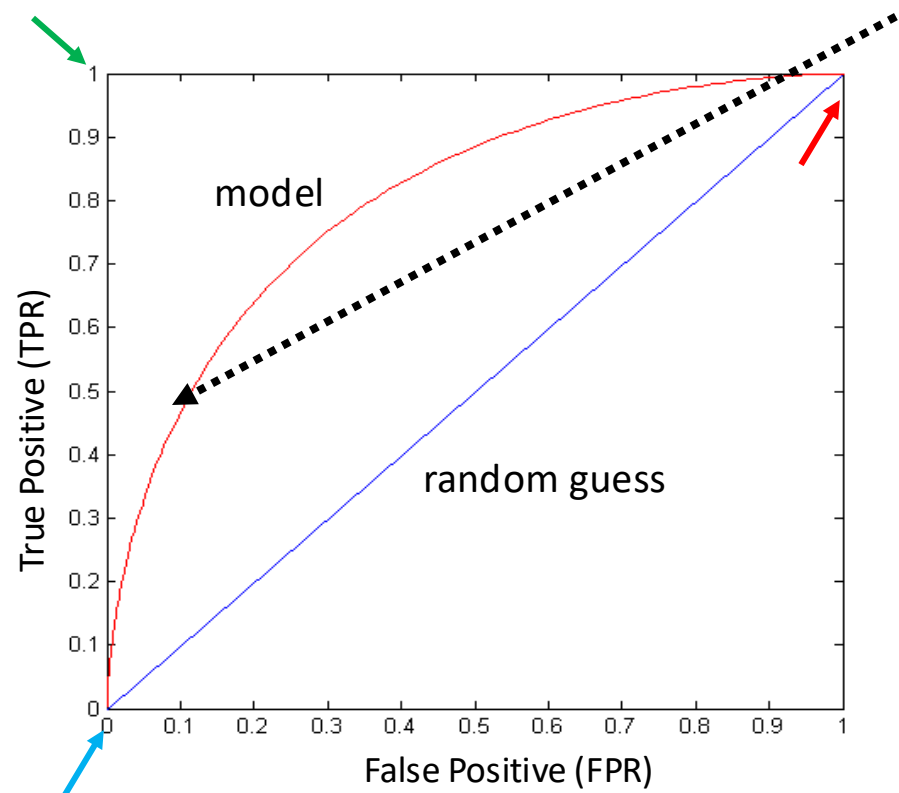
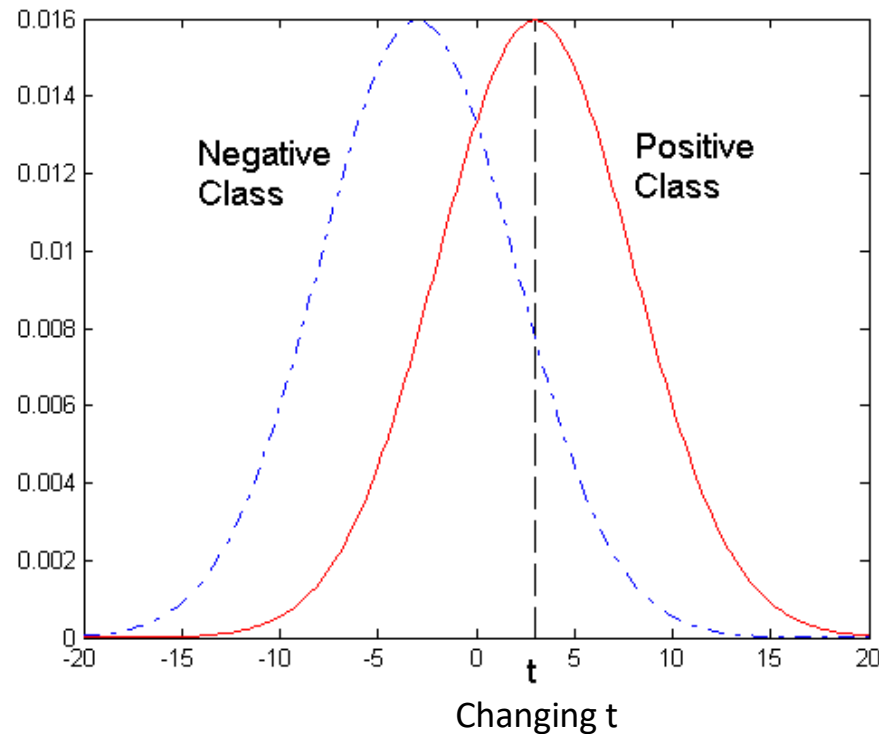
$$FPR = \frac{FP}{FP + TN}$$

What fraction of **negative instances** were predicted **incorrectly**?

	Prediction		
		Yes	No
Actual	Yes	a (TP)	b (FN)
	No	c (FP)	d (TN)

ROC curve

- 1-dimensional data set containing **2** classes (*positive* and *negative*)
- Any points located at $x > t$ is classified as *positive*



At threshold t :

TPR=0.5, FPR=0.12

$$TPR = \frac{TP}{TP + FN}$$

$$FPR = \frac{FP}{FP + TN}$$

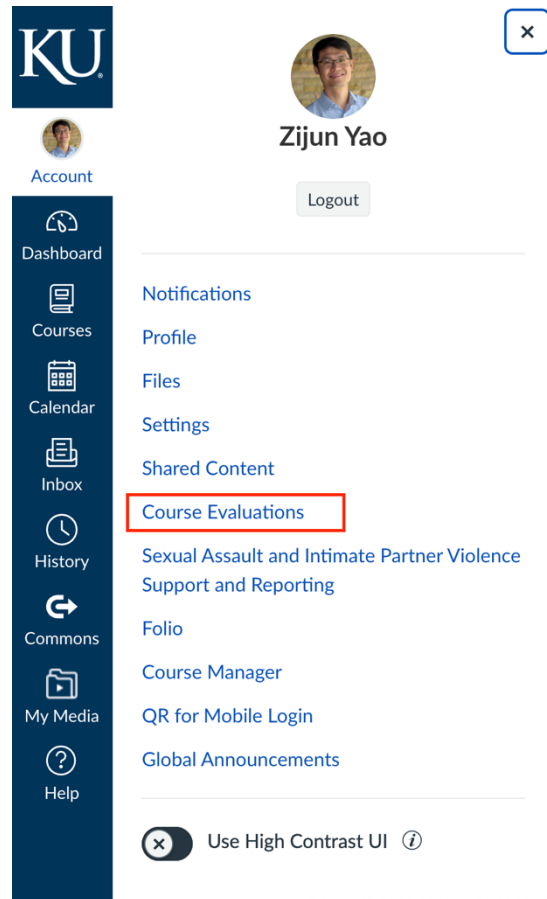
(TPR=0, FPR=0): Model predicts every instance to be a negative class

(TPR=1, FPR=1): Model predicts every instance to be a positive class

(TPR=1, FPR=0): The perfect model with zero misclassifications

Take Course Evaluation

- Your feedback is very important for the course (till 5/9)



Link: click "Course Evaluations" in "Account" of Canvas